POSM qFoundry

Autonomous Engineering Infrastructure

Product, Architecture, Security, Statistical Intelligence, Hardware, and Verification Specification

Version 0.1-DRAFT

June 2026

Photonics and Optics Student Makers (POSM)

Internal design baseline for review; not yet an implementation guarantee.

Document Control

Field	Value
Document	POSM qFoundry System Specification
Status	Draft design baseline
Version	0.1-DRAFT
Date	June 2026
Primary organization	Photonics and Optics Student Makers (POSM)
Working product name	POSM qFoundry
Primary deployment	POSM-managed lab installation with approved University of Minnesota users
Public deployment model	Free software distributed for local use with user-supplied model credentials
Normative keywords	**shall**, **must**, and **required** denote normative requirements; **should** denotes a strong recommendation; **may** denotes an option

Purpose of this specification

This document converts the qFoundry concept from an extended design conversation into a traceable engineering baseline. It defines the problem, operating model, user governance, supervisor–worker protocol, security boundaries, cost controls, Bayesian and machine-learning methods, desktop interface, server deployment, hardware strategy, and verification program. It is intended to be sufficiently explicit that a coding agent can implement bounded milestones without silently inventing product behavior.

The specification is deliberately broader than an ordinary software requirements document. qFoundry is simultaneously a multi-agent orchestration platform, a financial-control system, a security-sensitive execution environment, a durable workflow engine, a scientific telemetry platform, and a user-facing project-management application. Treating only the chat interface or only the coding loop as the product would miss the majority of the engineering risk.

Revision policy

No approved requirement is silently overwritten. Material changes are represented as versioned change requests with impact analysis, cost implications, security implications, affected acceptance tests, and

human approval. Raw telemetry and historical decisions remain immutable; corrections are appended as new records.

Current locked assumptions

- qFoundry begins as a private POSM system, then may be released publicly after several months of successful operation.
- The public software allows operators to provide their own OpenAI and Anthropic credentials and configure their own policies.
- The POSM deployment uses approved UMN accounts, one active project per ordinary user, centrally governed resources, and an initial maximum project budget of $30.
- Ordinary users approve their own product contract before submission, but a separate system intake review checks safety, budget, repository scope, and resource feasibility.
- OpenAI models act as project supervisors; Claude Sonnet or Opus performs coding work.
- A fifth logical pair, Sentinel, audits cost, progress, security, scheduling, estimation, and system improvement.
- The initial application is a polished Windows desktop client backed by local or laboratory-hosted services and Linux worker isolation.
- POSM may contribute up to approximately $1000 toward the first dedicated compute system; expensive components should be pursued through donations, surplus, sponsorship, refurbishment, or academic discounts.

How to read the document

Parts I and II define the product and core architecture. Part III defines security and reliability. Part IV defines deployment, hardware, and sustainability. Part V defines proof obligations and the implementation sequence. The appendices contain machine-readable schemas, matrices, examples, and research model cards intended to be used directly during implementation.

Contents

List of Figures

List of Tables

Part I

Vision, Scope, and Governance

Chapter 1

Executive Summary

POSM qFoundry is a privately hostable, eventually open-source engineering orchestration platform designed to multiply the output of a small technical organization without surrendering human control. A user begins with an idea or an existing repository. A project supervisor conducts a detailed discovery interview, compiles a versioned contract, generates a cost- and time-aware execution plan, and then manages a persistent Claude coding worker through repeated implementation, permission, verification, and correction cycles. The user does not need to manually return after every Claude response; the supervisor advances the project until a genuine human decision, budget boundary, security issue, or final acceptance point is reached.

The system is motivated by a simple operational bottleneck. Modern coding agents can independently inspect repositories, edit code, execute commands, run tests, and produce pull requests, but the human operator must still supervise one conversation at a time. When four unrelated POSM projects exist, the human often waits for one worker instead of directing the portfolio. qFoundry changes the human role from prompt courier to engineering director.

Design Principle

qFoundry shall automate routine supervision without automating away accountability. Every meaningful action must remain attributable to a contract requirement, a supervisor decision, a deterministic policy rule, a worker tool invocation, and observable evidence.

The principal logical unit is a **Cell**: one persistent OpenAI supervisor, one persistent Claude worker, one isolated execution environment, one repository context, one budget, and one project contract. Four development Cells may operate concurrently or be scheduled according to resource and budget constraints. A fifth Cell, Sentinel, ingests immutable portfolio telemetry and performs security auditing, Bayesian estimation, anomaly detection, scheduling analysis, model-routing evaluation, and controlled system-improvement experiments.

qFoundry is not an MCP server with a graphical skin. MCP may be used for portable tools, but the control plane requires explicit APIs, durable workflow state, a fixed-point financial ledger, policy enforcement, event sourcing, model gateways, container isolation, and human-approval protocols. OpenAI's Agents SDK provides sessions, guardrails, usage tracking, and tracing suitable for the supervisor layer, while Anthropic's Agent SDK provides a programmable coding agent loop with

filesystem and command tools suitable for the worker layer [5, 20, 21].

1.1 Major capabilities

The target system includes:

- contract-first project discovery with contradiction detection and change control;
- Claude-optimized task packs derived from the approved contract;
- autonomous multi-turn supervisor–worker execution;
- explicit green, yellow, and red permission tiers;
- live Antigravity-style execution streaming of prompts, responses, commands, tool outputs, diffs, tests, and costs;
- per-user, per-project, per-model, daily, and global budgets;
- preflight quotes with calibrated cost and time distributions;
- hard spending ceilings implemented through request reservation rather than optimistic after-the-fact alerts;
- durable pause, resume, crash recovery, and human-in-the-loop operation;
- Git branches, pull requests, checkpoints, rollbacks, and independent verification;
- email escalation and reply ingestion during configured communication windows;
- multi-user authorization with project isolation and master oversight;
- statistical portfolio intelligence grounded in registered models and empirical calibration;
- a portfolio digital twin for deadline, cost, and resource scenario analysis;
- an extensible desktop client and a dedicated laboratory server deployment path.

1.2 POSM operating policy

The POSM-hosted deployment begins with approved UMN accounts only. An ordinary user may have one active project and may choose any funded amount up to a $30 absolute project ceiling. Unused funds remain in the user's account for later allocation. Users see only their own project data. The master administrator sees the complete portfolio and may operate multiple projects. Deliberate abuse results in immediate account freeze and investigation; confirmed abuse results in permanent suspension.

The public software distribution is different. Anyone may install qFoundry, provide their own API credentials, and configure their own policies. POSM's account restrictions and wallet controls are deployment settings rather than artificial restrictions on the local application.

1.3 Budget philosophy

Financial control is a first-class safety property. qFoundry distinguishes a target, a user-selected tolerance, and an absolute ceiling. It stores money as integer microdollars or exact decimal values, never binary floating point. Before each provider request, the model gateway reserves a conservative maximum based on known input size, allowed output, provider pricing, tool charges, and required salvage reserve. If the request cannot fit, it is not started.

A project approaching its limit enters **salvage mode**: new feature work stops, the repository is

checkpointed, the cheapest useful verification is run, unresolved items are documented, the session is made resumable, and the user receives a report. The platform therefore fails gracefully rather than consuming its final cents mid-edit.

1.4 Deployment strategy

The first implementation runs on a Windows computer as a Tauri desktop application with local services and Linux execution through WSL2 or containers. Tauri supports a web-technology frontend with native desktop packaging and explicit capabilities [23, 24]. The eventual POSM installation moves the orchestration backend to a dedicated Linux workstation in McLeod's lab, while authorized users connect through the same desktop client over private networking.

Because OpenAI and Anthropic perform model inference in their own data centers, the dedicated computer primarily needs CPU cores, large RAM, high-endurance NVMe storage, reliable networking, backup, and power protection. A large local GPU is optional unless qFoundry later runs local models, vision workloads, or GPU-accelerated scientific jobs.

1.5 Success criterion

The first release is successful only if one supervisor can direct one Claude worker through at least five consecutive coding cycles, expose every material action live, respect a tiny hard budget, survive a forced restart, deny sandbox and secret-access tests, accept a validated email reply exactly once, detect a deliberately false completion claim, and deliver a real bounded project that satisfies its contract. The system scales to two and then four project Cells only after that central loop is demonstrated.

Chapter 2

Motivation and Problem Statement

2.1 The human-supervision bottleneck

Coding agents have made implementation dramatically faster, but their productive use still requires repeated human intervention. A typical session proceeds as follows: the user supplies a detailed prompt; the agent explores a repository; the agent requests permission; the user approves; the agent edits and tests; the agent reports completion; the user inspects the result and writes the next prompt. This loop is effective for one project but does not scale to a portfolio.

The wasted resource is not only elapsed time. It is human attentional context switching. A technical lead who supervises four projects manually must remember four architectures, four unfinished conversations, four budget states, and four sets of unresolved decisions. Valuable work pauses whenever the lead is in class, in the laboratory, asleep, or simply focused on another task.

qFoundry formalizes and automates the middle of this loop. The human invests deeply at the beginning, when product intent matters most, and at explicit decision boundaries. A supervisor maintains continuity between those boundaries.

2.2 Why simple parallel chats are insufficient

Running several coding chats simultaneously solves only execution concurrency. It does not solve:

- ensuring each worker remains aligned with an approved product vision;
- deciding the next task when a worker finishes;
- evaluating whether a completion claim is true;
- approving safe tool use while escalating dangerous actions;
- preventing cost overruns;
- recovering from crashes and stalled sessions;
- preserving a trustworthy audit trail;
- allocating finite machine resources across users;
- learning which model and workflow are economical for each task.

A robust system must therefore treat the agent as a controlled process, not a magical coworker.

2.3 Organizational value to POSM

POSM develops scientific hardware and software across photonics, RF, FPGA control, optomechanics, simulation, web systems, and documentation. These efforts contain substantial coding work that is important but often laborious: frontend development, data models, test harnesses, device-control layers, numerical validation, component databases, build systems, and documentation.

qFoundry can create several forms of leverage:

1. **Parallel execution.** Independent projects can progress without requiring one person to manually shuttle prompts.
2. **Higher specification quality.** Supervisors force users to define workflows, edge cases, non-goals, and acceptance criteria before money is spent.
3. **Institutional memory.** Contracts, decisions, evidence, and reusable skills survive student turnover.
4. **Cost visibility.** The organization can understand which kinds of projects consume money and which routing strategies are effective.
5. **Training.** Students can observe professional engineering workflows, tests, permissions, architecture decisions, and review evidence in real time.
6. **Shared infrastructure.** A dedicated laboratory machine can turn otherwise idle compute into a controlled queue of engineering work.

2.4 Why trust requires observability

A user trusts an interactive coding agent partly because the work is visible: files are read, commands run, errors appear, tests pass, and the agent reacts. A black-box orchestrator that displays only a spinner and a final message would destroy that trust.

qFoundry therefore distinguishes **activity**, **progress**, and **confidence**. Activity means the worker is doing something. Progress means measurable contract obligations are closer to satisfaction. Confidence describes the strength and calibration of evidence. High activity with no passing tests or resolved requirements is not progress.

Normative Requirement

Every project view shall expose a live execution stream containing the supervisor's prompts, the worker's visible responses, tool calls, commands, command output, file changes, tests, permissions, cost deltas, and progress evidence. Every summary metric shall link to its source events.

2.5 Why the system must be financially intelligent

Agentic coding can burn money through repeated exploration, overlong context, unnecessary strong-model use, or failure loops. The system should not merely display cost after the fact. It should choose economical execution strategies, estimate uncertainty, reserve funds, recognize diminishing returns, and stop when further work is not worth the expected value.

This motivates the local model gateway, preflight quote, portfolio optimizer, context packs, prompt caching, discounted batch work, model routing, and Sentinel's statistical evaluation. OpenAI documents

prompt caching and discounted asynchronous batch processing as mechanisms that can materially lower repeated-context and nonurgent costs [18, 19].

2.6 Non-objectives

qFoundry is not intended to:

- remove human ownership of product direction;
- permit unbounded autonomous spending;
- grant coding workers unrestricted access to the host computer;
- automatically merge to protected branches;
- deploy production systems without explicit approval;
- replace a version-control platform or full integrated development environment;
- display or claim access to hidden chain-of-thought;
- produce statistically impressive but unvalidated "AI confidence" numbers;
- guarantee that every project can be completed within a small budget.

Chapter 3

Product Scope and Deployment Models

3.1 Product identity

The working public name is POSM qFoundry. "Foundry" communicates that user ideas, contracts, repositories, model labor, verification, and infrastructure are transformed into engineering artifacts. The lowercase "q" preserves the POSM naming language without implying that the application itself performs quantum computation.

The preferred tagline is:

Autonomous Engineering Infrastructure

3.2 Two deployment classes

qFoundry shall support two operating classes.

3.2.1 Independent local deployment

An individual or laboratory downloads the software, supplies its own OpenAI and Anthropic credentials, chooses its own providers, sets its own budgets, and runs the application locally or on privately controlled infrastructure. The project shall not hardcode POSM's user caps or institutional policy into the core engine.

3.2.2 POSM managed deployment

The POSM instance is a shared service. It adds:

- approved UMN accounts;
- master, reviewer, and ordinary user roles;
- centrally funded or user-funded wallet accounting;
- one active project per ordinary user;
- an initial $30 hard maximum per project;
- resource quotas;
- safety intake review;

- organization-wide Sentinel monitoring;
- institutional retention and incident-response policy.

3.3 Private-first release sequence

The software remains private to POSM for an initial verification period. The private phase is not intended to create permanent vendor lock-in; it permits instrumentation, red-team testing, budget calibration, interface refinement, and correction of security failures before public distribution.

A public release requires at minimum:

1. successful completion of the verification gates in chapter 27;
2. documented installation and configuration;
3. a stable migration path for the database and contracts;
4. externalized deployment policy;
5. removal of private POSM secrets and infrastructure assumptions;
6. threat-model review;
7. license and trademark policy approval.

3.4 Project types

A project may begin as:

- a new idea with no repository;
- a new project that will later be pushed to GitHub;
- an existing GitHub repository;
- a bounded feature or bug-fix request in an existing repository;
- a standalone artifact such as a command-line tool, desktop application, analysis package, document generator, or simulation module.

The architecture should not assume that every project ends as a pull request. A deliverable package may include a repository, pull request, ZIP archive, executable, installer, report, dataset, screenshots, verification evidence, or reproducibility instructions.

3.5 Hosted-service acceptable-use scope

The POSM deployment is an academic research and educational resource. Initial prohibited uses include:

- malware, credential theft, persistence, evasion, destructive payloads, or unauthorized access;
- projects primarily intended for personal commercial profit through subsidized POSM compute;
- autonomous financial trading, investment execution, or financial-advice systems;
- medical diagnosis, treatment recommendation, clinical decision support, or other unreviewed high-stakes health systems;
- weapons development or optimization;
- privacy-invasive surveillance, doxxing, stalking, or unauthorized biometric identification;

- production control of safety-critical physical systems;
- illegal activity or attempts to bypass institutional policy;
- projects designed to attack qFoundry, other users, or external systems except within an explicitly authorized canary/red-team environment.

The public software license cannot meaningfully prevent a local operator from changing policy; the restrictions above govern the POSM-hosted service.

3.6 Scope boundaries for version 0.1

Version 0.1 shall include the architectural skeleton for multi-user operation but may expose only one master account and one test-user account initially. It shall support five logical project slots, a maximum of two active Claude workers by default, and an advisory Sentinel. It shall not perform autonomous production deployment or automatic protected-branch merging.

Locked Design Decision

Version 0.1 is intentionally maximal in observability and governance but bounded in execution breadth. The system should feel powerful because it exposes the complete engineering process, not because it prematurely implements every possible cloud, mobile, and self-improvement feature.

Chapter 4

Users, Roles, and Governance

4.1 Roles

4.1.1 Master administrator

The master administrator may:

- view all projects, budgets, telemetry, contracts, security findings, and resource use;
- run multiple projects concurrently subject to global limits;
- approve accounts, red permissions, budget changes, deletions, and protected-branch merges;
- pause, quarantine, or terminate any project;
- configure organization-wide policies and model access;
- review Sentinel interventions and improvement proposals.

4.1.2 Ordinary user

An ordinary user may:

- maintain one active project at a time;
- conduct discovery and approve the project's product contract;
- allocate funding from the user's available balance up to the deployment cap;
- view all execution data associated with that user's project;
- approve yellow actions that concern only the user's repository, data, and pre-authorized integrations;
- reply to ordinary supervisor questions by email;
- accept or reject technically ready deliverables;
- request deletion or export of project data.

An ordinary user may not view another user's project, increase organization-wide limits, approve red actions, modify safety invariants, or merge protected branches without the required second confirmation.

4.1.3 Reviewer or faculty role

A reviewer has read access to designated projects and may provide technical or product approval. This role is intended for faculty advisers, senior POSM members, or designated domain experts. A reviewer does not automatically receive master-level financial or security authority.

4.1.4 Service identities

Supervisor, worker, verifier, Sentinel, model gateway, credential broker, scheduler, email service, and GitHub App are separate service identities. No service identity inherits a human's complete privileges.

4.2 Account lifecycle

Only approved UMN accounts may use the POSM-hosted deployment. Registration is a request followed by administrator approval. Authentication should use the University's Google Workspace identity or another institution-approved federated method, while qFoundry maintains its own account allowlist and role mapping.

Account states include:

$$\mathsf{Requested} \rightarrow \mathsf{Approved} \rightarrow \mathsf{Active} \rightarrow \mathsf{Frozen} \rightarrow \mathsf{Suspended/Deleted}$$

A credible deliberate-abuse signal causes immediate freeze, not a warning period. Because automated security systems can produce false positives, evidence is preserved and an administrator reviews the incident before irreversible deletion. Confirmed deliberate abuse results in permanent suspension and appropriate institutional reporting.

4.3 Project ownership and visibility

Every event, artifact, decision, budget entry, contract version, repository credential, and email escalation belongs to exactly one project and one tenant boundary. Ordinary users can subscribe only to event streams for projects in which they are members. Project isolation shall be enforced in the API authorization layer, database policies, credential broker, object storage paths, event subscriptions, and container mounts; hiding interface elements is not security.

4.4 Project submission and intake

A user completes discovery, reviews the compiled contract, chooses funding, and explicitly selects **Send to Foundry**. Submission does not immediately spend money. A separate intake gate verifies:

- acceptable-use compliance;
- repository ownership and scope;
- requested network and integration access;
- budget feasibility;
- current system capacity;
- absence of obvious contract contradictions;
- presence of minimum acceptance criteria.

The intake gate does not override the user's product preferences. It approves whether the system may execute the contract under current policy.

4.5 Financial ownership

A user's unused balance remains associated with that user. Sentinel may recommend that the user allocate more money or that POSM supply institutional support, but it cannot move funds between users. Future payment integration shall maintain a ledger of deposits, reserved funds, actual spend, refunds, expiration, and adjustments.

4.6 Two-step protected-branch control

No AI merges to a protected branch automatically. The merge workflow requires:

1. user acceptance of the reviewed deliverable;
2. a separate confirmation displaying the exact repository, source branch, target branch, commit hash, verification status, and change summary.

This two-step design reduces accidental merges caused by ambiguous natural-language approval.

Chapter 5

User Journeys

5.1 Journey 1: starting from an idea

1. The user selects **New Project** and describes the desired outcome in ordinary language.
2. The supervisor enters discovery mode and asks structured follow-up questions about users, workflows, inputs, outputs, edge cases, technology constraints, accuracy, UI expectations, non-goals, budget, deadline, and acceptance evidence.
3. The supervisor maintains a visible completeness map and identifies contradictions or unresolved high-impact decisions.
4. The contract compiler produces a human-readable contract, machine representation, milestone graph, acceptance plan, and Claude execution pack.
5. The user edits or rejects any part until satisfied.
6. The user approves the contract and selects a funded amount up to the project maximum.
7. The system performs safety and feasibility intake.
8. The scheduler produces alternative execution plans and the user selects or accepts the recommended plan.
9. The first milestone begins in an isolated repository created for the project.
10. When the project becomes technically ready, the user reviews deliverables, spends remaining budget on requested changes, or accepts and closes the project.

5.2 Journey 2: connecting an existing repository

The user authorizes the qFoundry GitHub App for a selected repository. The credential broker obtains a short-lived installation token restricted to that repository and the required permissions. GitHub documents that installation tokens can be limited to selected repositories and permissions and expire after a short period [11, 12].

The system indexes the repository, runs read-only baseline checks, identifies languages and test commands, and produces a repository map. The supervisor then conducts discovery focused on the requested change rather than re-specifying the entire product.

5.3 Journey 3: routine permission

Claude requests installation of a common schema-validation package from an approved registry. Deterministic policy verifies registry, package identity, pinned version, license, known vulnerabilities, and installation-script risk. The request is automatically approved and an audit event is sent to the supervisor. The user sees the action in the live stream but is not interrupted.

5.4 Journey 4: ambiguous architecture decision

Claude discovers two valid designs with meaningful product consequences. The worker reports the alternatives rather than choosing silently. The supervisor consults the contract, existing decisions, budget, deadline, and architecture registry. If the decision is not reserved to the user and one option is clearly dominant, the supervisor may decide. Otherwise it creates an escalation bundle containing options, tradeoffs, recommendation, cost, schedule impact, and a default if the user delays.

5.5 Journey 5: email decision

During the configured communication window, qFoundry sends an email with a unique signed escalation identifier. The user replies from an authorized address. Gmail push notifications can inform the backend of mailbox changes without constant polling, and Gmail threads preserve the conversation context [13, 15].

The supervisor interprets the reply and records the interpretation. Ordinary project decisions may resume automatically. Red actions or consequential merges require a secure dashboard confirmation of the interpreted instruction.

5.6 Journey 6: budget exhaustion

At 92 percent of the funded amount, the supervisor receives a projected-completion warning and stops beginning unquoted major work. At 95 percent, the project enters salvage mode. The worker finishes or safely interrupts the current bounded command, the system runs the cheapest useful checks, creates a checkpoint, saves session identifiers, summarizes remaining work, and emails the user. The user may allocate more of the user's available balance or close the project.

5.7 Journey 7: master takeover

The master administrator opens a project, presses **Pause after current command**, inspects the live terminal and diff, sends a direct instruction to Claude or the supervisor, and returns control. All intervention is logged. A global emergency stop terminates worker processes, disables new provider calls, revokes ephemeral credentials, and places active projects into recoverable quarantine.

5.8 Journey 8: public local user

An external user downloads qFoundry, selects local single-user mode, supplies provider credentials, chooses a storage directory, and configures budget policy. POSM's UMN allowlist, centralized wallet, one-project cap, and institutional retention policy are not enabled unless the operator explicitly installs the managed-deployment profile.

Part II

Core System Architecture

Chapter 6

System Architecture

6.1 Architectural overview

qFoundry is divided into a control plane, execution plane, data plane, integration plane, and presentation plane.

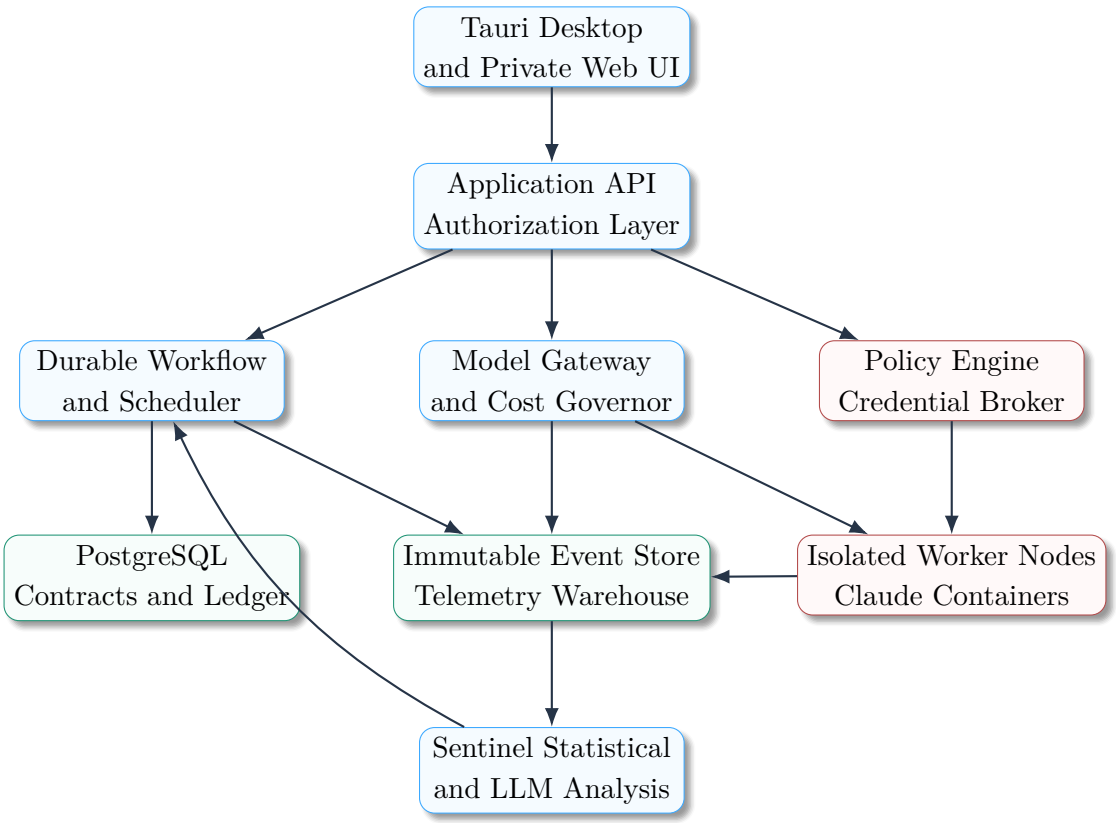

Figure 6.1: High-level qFoundry architecture. The LLMs are components inside a larger deterministic control system.

6.2 Control plane

The control plane contains user authorization, project state, contracts, budgets, scheduling, permissions, approvals, model routing, and policy. It decides what may happen. It does not execute arbitrary worker shell commands directly.

6.3 Execution plane

The execution plane contains Claude worker sessions, verification runners, build tools, browsers, and project containers. Each active project has an isolated execution identity, filesystem boundary, network policy, resource envelope, and ephemeral integration credentials.

6.4 Data plane

The data plane stores contracts, project metadata, budgets, model usage, workflow state, immutable events, artifacts, test reports, diffs, screenshots, and security incidents. High-volume raw logs may have shorter retention than decisions and financial records.

6.5 Integration plane

The integration plane contains provider APIs, Gmail, GitHub, package registries, Tailscale, optional local models, and future MCP servers. All integrations pass through explicit adapters and are attributed to a project identity.

6.6 Presentation plane

The desktop application is the primary operator interface. The same frontend components may render a private browser interface for remote access. The interface is information-dense but layered through expandable detail, density modes, and evidence-linked metrics.

6.7 Event-driven operation

Supervisors shall not poll continuously or generate idle tokens. They wake on events such as:

- worker response completed;
- permission requested;
- worker question or blocker raised;
- verification failed;
- budget threshold crossed;
- human email or dashboard response received;
- scheduled checkpoint reached;
- Sentinel anomaly detected.

This event-driven design minimizes cost and makes the system reconstructable from an event history.

6.8 Durable workflows

Human decisions can take minutes or days. The workflow runtime must persist state without holding an expensive model or fragile process open. Temporal provides durable workflows, signals, timers, and audit history for human-in-the-loop systems [25, 26]. Activities that cause external side effects must be idempotent because workflow engines may retry them after failure.

6.9 Service boundaries

The model gateway, credential broker, worker runner, and user-facing API shall be separate processes or trust domains. A worker's shell environment must never inherit provider API keys or master credentials. The dashboard shall never make direct provider calls.

6.10 Multi-node future

The architecture supports a heterogeneous pool of x86 worker nodes. A high-memory primary node may run demanding repositories while donated workstations run lighter jobs. Node labels describe architecture, RAM, CPU, GPU, operating system, approved networks, and installed toolchains. The scheduler chooses a compatible node rather than assuming one giant tower.

Chapter 7

Supervisor–Worker Protocol

7.1 Cell definition

A qFoundry Cell contains:

- one persistent project supervisor using an OpenAI reasoning model;
- one persistent Claude Sonnet or Opus coding session;
- one approved contract version;
- one current milestone and task graph;
- one budget allocation and cost ledger;
- one isolated repository environment;
- one permission policy;
- one event stream and evidence store.

The supervisor and worker have different responsibilities. The supervisor owns interpretation of the contract, milestone sequencing, completion review, escalation, and economical routing. Claude owns repository exploration, implementation, command execution, tests, and technical reporting.

7.2 Closed-loop sequence

1. The supervisor compiles a task-specific execution pack from the approved contract.
2. The model gateway verifies available budget and reserves maximum call cost.
3. Claude receives the task and works inside the assigned environment.
4. Tool calls are checked by deterministic policy; ambiguous requests are routed to the supervisor or human.
5. Claude emits streamed responses, tool events, file changes, and command results.
6. The worker finishes a turn with a structured work report.
7. Deterministic verification runs before the supervisor spends tokens reviewing prose.
8. The supervisor compares repository state and evidence with requirements.
9. The supervisor either sends a corrective prompt, advances the milestone, opens a change request, or escalates to the user.
10. The loop stops at completion, budget boundary, human decision, security pause, or operator interruption.

7.3 Worker completion report

Claude's final message for a turn shall include machine-parseable fields:

```
{
  "summary": "Implemented graph serializer with circular-reference support",
  "changed_files": ["src/serializer.ts", "tests/serializer.test.ts"],
  "commands": [
    {"command": "npm test -- serializer", "exit_code": 0}
  ],
  "requirements_addressed": ["REQ-GRAPH-014"],
  "known_limitations": [],
  "questions": [],
  "claimed_status": "task_complete"
}
```

The report is a claim, not proof. qFoundry independently captures Git, command, and test evidence.

7.4 Supervisor decision record

A supervisor response shall be structured:

```
{
  "assessment": "incomplete",
  "evidence": [
    "unit tests pass",
    "required reconnect integration test is absent"
  ],
  "contract_refs": ["REQ-NET-009"],
  "next_action": "continue_worker",
  "next_prompt": "Add the missing reconnect test and verify retry limits.",
  "milestone_complete": false,
  "requires_human": false,
  "estimated_next_cost_usd": "0.42"
}
```

7.5 False completion resistance

A supervisor shall never accept "finished" based solely on the worker's message. It examines:

- Git status and diff;
- changed-file scope;
- build, lint, type-check, and test results;
- required artifacts;
- contract traceability;
- security and dependency scans;
- reviewer findings at designated milestones.

7.6 Model selection

Claude Sonnet is the default implementation model. Opus is reserved for tasks with evidence of higher difficulty, such as complex architecture, numerically sensitive algorithms, repeated diagnosed failures, security-critical code, or cross-module debugging. A failed Sonnet attempt does not automatically justify Opus; the supervisor first determines whether the specification, environment, or test harness is the true problem.

7.7 Interrupt and takeover

The operator may pause the supervisor, pause Claude, stop after the current command, interrupt immediately, send a direct instruction, inspect the worktree, fork the approach, roll back, change worker model, or hand control back. The system records when normal automation was bypassed and what state was restored afterward.

7.8 Visible reasoning boundary

The application displays prompts, responses, decisions, evidence, tool calls, and concise stated rationales. It does not claim to reveal hidden private reasoning. This boundary preserves honest observability while still exposing the complete engineering record needed for oversight.

Chapter 8

Discovery, Contract, and Contract Compiler

8.1 The contract as north star

The approved contract is the highest project-specific authority. The supervisor may choose implementation details within it but may not silently alter intended behavior, scope, user workflow, safety constraints, or acceptance criteria. Material deviations require a change request.

8.2 Discovery mode

Discovery is intentionally thorough. The supervisor should challenge vague phrases such as "make it intuitive," "make it accurate," or "support everything." Questions cover:

- intended users and permissions;
- primary and secondary workflows;
- exact inputs, outputs, units, formats, and error behavior;
- required integrations and repository boundaries;
- UI density, responsiveness, and accessibility;
- performance, reliability, security, and privacy constraints;
- MVP and future scope;
- non-goals and forbidden changes;
- acceptance evidence and human review points;
- deadlines, budgets, and model policies.

The supervisor displays a contract-readiness assessment with explicit unresolved questions. It shall not present a high score simply to encourage progress.

8.3 Requirement structure

Each requirement has a stable identifier, title, rationale, priority, behavior, owner milestone, dependencies, acceptance method, required evidence, and status. Example:

Normative Requirement

`REQ-COST-014`: The project view shall display worker and supervisor spending separately. A user shall be able to select any displayed amount and inspect the provider usage events, pricing version, reconciliation state, and calculation that produced it.

8.4 Contradiction detection

The compiler performs deterministic and model-assisted checks for:

- mutually exclusive permissions;
- deadlines incompatible with budgets;
- required features excluded by non-goals;
- acceptance tests that do not observe the claimed behavior;
- security policy incompatible with an integration;
- undefined ownership of shared components;
- duplicate or circular requirements;
- terms used with inconsistent meanings.

Model-assisted findings are suggestions until validated by a deterministic rule or human decision.

8.5 Contract outputs

The compiler produces:

1. a human-readable specification;
2. a machine-readable contract document;
3. a requirement dependency graph;
4. an acceptance-test plan;
5. a milestone plan;
6. a traceability matrix;
7. task-specific Claude execution packs;
8. prompt-cacheable stable context packs;
9. a list of unresolved assumptions and human-reserved decisions.

8.6 Claude optimization

The execution pack places stable authority and project essence first, followed by milestone and task-specific information. This layout supports prompt caching because repeated exact prefixes can be reused by providers [19]. Only relevant contract sections are included in a task. The pack is versioned and hashed so the dashboard can show exactly what Claude received.

8.7 Change requests

A change request contains:

- requested behavior change;
- reason and requester;
- affected requirements, tests, milestones, and architecture;
- estimated cost and schedule effect;
- security and data impact;
- supervisor recommendation;
- human decision;
- resulting contract version.

No model may approve a substantive change request on behalf of the user.

8.8 Requirements traceability

The canonical chain is:

$$\text{Requirement} \rightarrow \text{Milestone} \rightarrow \text{Task} \rightarrow \text{Commit} \rightarrow \text{Test/Artifact} \rightarrow \text{Review} \rightarrow \text{Acceptance}$$

A project cannot be marked technically ready while mandatory requirements lack evidence or have unresolved failing tests.

Chapter 9

Workflow Runtime and State Machines

9.1 Project state model

A project moves through explicit states rather than an unbounded chat:

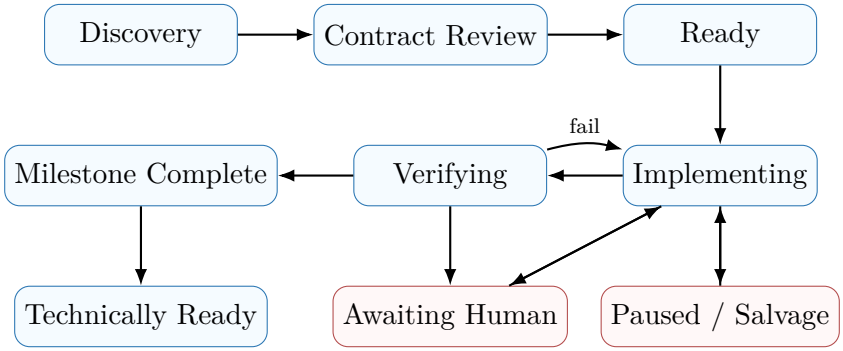

Figure 9.1: Simplified project state machine. Substates record the current task, worker session, permission wait, and verification stage.

9.2 Durability requirements

A project may wait overnight for a user, survive an application restart, or recover from a worker-node crash. The durable workflow shall persist:

- current state and substate;
- contract and milestone versions;
- Claude session identifier and restart instructions;
- pending provider call reservations;
- current worktree and checkpoint;
- unresolved permission and human decisions;
- retry counters and idempotency keys;
- next scheduled wake time.

Temporal is a recommended implementation because it supports signals, durable timers, human waits, and replayable state [25, 26]. The architecture shall nevertheless abstract the workflow engine so the

core state model is not embedded only in Temporal-specific code.

9.3 Idempotency

The following operations require stable idempotency keys:

- sending an email;
- creating a Git branch or pull request;
- charging or reserving a wallet entry;
- applying a permission decision;
- creating an artifact;
- recording a provider usage response;
- executing any external integration with side effects.

Retries may repeat internal computation, but they shall not duplicate an external action.

9.4 Pause semantics

qFoundry distinguishes:

Pause new work stop starting new tasks while allowing safe current operations to finish;
Stop after command allow the current command to exit, then checkpoint;
Interrupt now send an interrupt to the worker process and preserve partial state;
Quarantine revoke credentials and network access, freeze the worktree, and await security review;
Emergency stop disable all new provider calls and terminate active worker processes.

9.5 Retry policy

Provider and network errors may be retried with bounded exponential backoff. Logic errors, repeated failing tests, permission denials, and budget failures are not blindly retried. A third similar failed approach requires diagnosis, rollback, fresh session, model change, reviewer, or human escalation.

9.6 Schedule-aware waits

The workflow engine can defer expensive or nonurgent work until a cheaper or operationally convenient period. Examples include overnight full test suites, discounted batch analysis, or resuming after the daily budget resets. The schedule is recalculated whenever costs, deadlines, human availability, or task outcomes change.

Chapter 10

Model Gateway, Routing, and Context Management

10.1 Purpose of the gateway

All OpenAI, Anthropic, and optional local-model traffic shall pass through a qFoundry-controlled gateway. The gateway is not a local replacement model. It is the policy and accounting boundary that prevents individual services from making ungoverned provider calls.

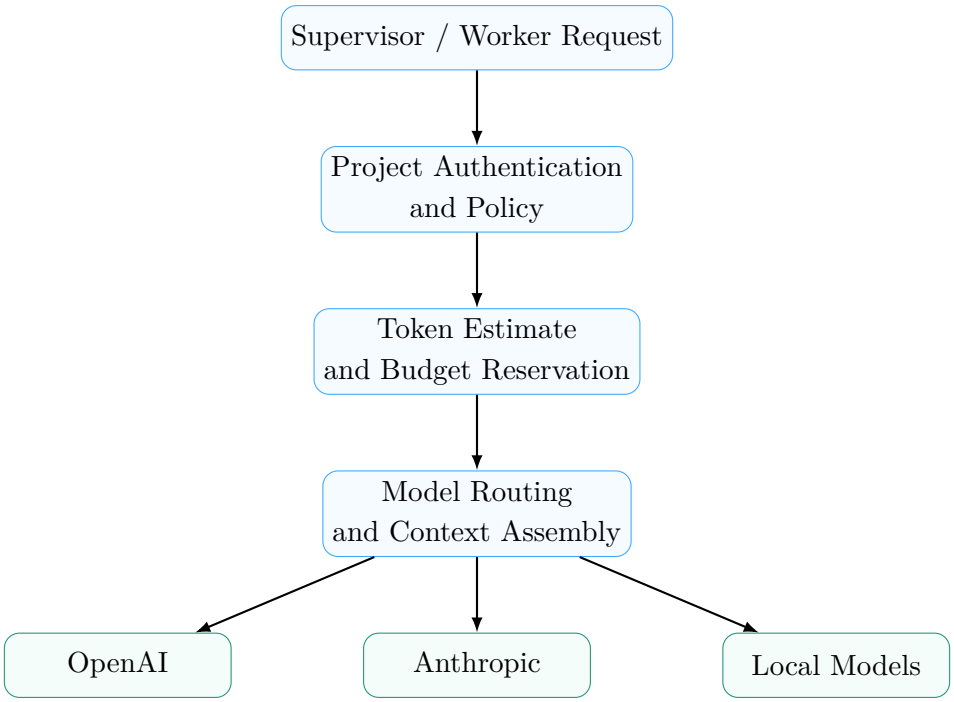

Figure 10.1: The model gateway centralizes authentication, budgets, routing, redaction, and usage attribution.

10.2 Gateway functions

The gateway shall provide:

- project and actor authentication;
- model allowlists and deny rules;
- exact pricing-table versioning;
- input token counting or conservative estimation;
- maximum-output enforcement;
- fixed-point budget reservation;
- request and response redaction;
- provider retry and rate-limit behavior;
- prompt-cache optimization;
- usage and latency events;
- fallback only when contract and policy permit;
- emergency revocation of provider access.

Anthropic documents centralized LLM gateways as a method for consistent routing, authentication, logging, and cost control for Claude Code deployments [6].

10.3 Model tiers

Tier	Typical work	Constraints
Deterministic	arithmetic, policy evaluation, test parsing, event routing	must be used when no model judgment is needed
Local small model	log labeling, routine summarization, duplicate detection, first-pass anomaly triage	no authority for consequential decisions
Cheap cloud model	formatting, low-risk classification, digest generation	bounded context and no red approvals
Strong OpenAI supervisor	discovery, contract review, architecture judgment, completion assessment, escalation	event-driven only
Claude Sonnet worker	normal implementation, tests, UI, documentation, refactoring	default coding model
Claude Opus worker	difficult architecture, numerical work, security-critical debugging, repeated diagnosed failure	explicit trigger and budget authorization
Independent reviewer	adversarial milestone review	fresh context; no assumption of prior success

Table 10.1: Initial model-routing tiers. Specific model identifiers remain configuration rather than contract text.

10.4 Minimal sufficient action

Before invoking a model, the gateway asks whether the event can be handled by deterministic logic, stored state, or a cheaper tier. A model shall not be paid to compare two decimal values, apply a known policy rule, or parse a test exit code.

10.5 Context packs

Stable and dynamic context are separated into:

- safety and authority pack;
- product-contract pack;
- architecture pack;
- repository map;
- coding standards;
- current milestone;
- recent changes;
- active failure or decision.

The system includes only relevant packs and stores the hash of each assembled request. Exact stable prefixes improve provider prompt-cache reuse. OpenAI documents automatic prompt caching for repeated prefixes and significant potential latency and input-cost reductions [19].

10.6 Nonurgent processing

Nightly summaries, historical replay, bulk evaluation, repository indexing, and portfolio analytics may use discounted asynchronous modes. OpenAI's Batch API provides lower-cost processing for jobs that can tolerate a longer completion window [18]. Such work shall never be placed in a batch if an interactive user decision or safety response depends on it.

10.7 Local models

A local model is optional in the first deployment and beneficial for high-volume low-risk work. Local output is treated as untrusted and evaluated like any other model. The system does not route coding work to a weak local model merely because local tokens appear free; power, latency, quality, and rework still have costs.

Chapter 11

Financial Ledger, Budgets, and Preflight Quotes

11.1 Budget objects

qFoundry supports:

- user wallet balance;
- project funded amount;
- supervisor, worker, reviewer, Sentinel, and tool sub-budgets;
- daily, monthly, provider, and model-specific limits;
- target, tolerance, and absolute ceiling;
- reserve, pending, estimated, and reconciled balances.

The initial POSM policy allows an ordinary user one active project with a maximum funded amount of $30. The software core allows operators of independent deployments to configure different policies.

11.2 Money representation

All internal money is represented as integer microdollars or exact decimal values with a fixed scale. Binary floating point is forbidden for wallet, reserve, spend, and reconciliation calculations. Every cost entry records currency, provider, model, pricing-table version, token categories, timestamp, estimate status, and authoritative reconciliation status.

11.3 Target and tolerance

A project budget may be configured as:

```
target_usd = 20.00
allowed_positive_tolerance_usd = 0.50
absolute_ceiling_usd = 20.50
```

Modes include:

Never exceed the absolute ceiling equals the target;
Target with tolerance the scheduler aims for the target but may use the explicit tolerance;
Approval on overrun the project pauses before expected target overrun and asks the user whether to use remaining funded balance.

11.4 Reservation accounting

Before a provider call, qFoundry computes a conservative upper bound:

$$R = C_{\text{input}} + C_{\text{max output}} + C_{\text{cache}} + C_{\text{tools}} + M, \tag{11.1}$$

where M is a policy margin. The request starts only if

$$S_{\text{reconciled}} + S_{\text{pending}} + R + R_{\text{salvage}} \leq B_{\text{hard}}. \tag{11.2}$$

After the response, actual provider usage replaces the reservation and unused value returns to the available balance.

Caution

The final cost of a streaming model response is not known until the response terminates. Therefore qFoundry can guarantee a configured hard ceiling only by reserving a maximum and refusing calls that do not fit. It cannot honestly promise to stop at exactly the token that makes a project cost $20.00.

11.5 Thresholds

Default POSM thresholds are:

Consumption	Behavior
0–70 percent	normal routing
70–80 percent	economy adjustments and forecast refresh
80–88 percent	no unquoted major tasks
88–92 percent	completion and salvage planning
92–95 percent	bounded current work only
95 percent	salvage mode
100 percent	absolute stop; no provider request may begin

11.6 Preflight quote

A preflight quote is a versioned probabilistic plan, not a sales promise. It contains:

- p10, p50, and p90 cost and duration estimates;
- probability of remaining below target and ceiling;
- success probability and calibration class;

- expected human decisions;
- model routing and concurrency;
- critical path;
- assumptions and uncertainty drivers;
- proof-before-build experiments;
- quote expiration and conditions for recalculation.

Alternative plans include cheapest feasible, recommended balanced, fastest reasonable, and highest-confidence.

11.7 Time-dependent planning

Urgency does not necessarily change provider token prices, but it changes execution strategy. A same-day deadline may require parallel workers, more frequent supervision, stronger models, less batching, and earlier reviewer invocation. A relaxed deadline can schedule overnight verification, discounted asynchronous analysis, cache-friendly sequential work, and waiting for dependencies.

11.8 Salvage mode

Salvage mode performs:

1. no new feature initiation;
2. safe completion or interruption of the current command;
3. repository checkpoint and session persistence;
4. targeted inexpensive verification;
5. requirement status and blocker summary;
6. exact ledger and forecast report;
7. next recommended prompt and resume instructions;
8. user notification.

11.9 User-visible cost information

Ordinary users see project funded amount, spend, remaining balance, forecast, and category breakdown. They need not see provider pricing formulas by default. The master and forensic views expose provider token counts, cache categories, pricing versions, estimates, reconciliation, and cost per task. Every displayed total identifies whether it is live estimated or provider reconciled.

Chapter 12

Portfolio Scheduling and Digital Twin

12.1 Scheduling problem

The portfolio governor allocates workers, model calls, CPU, RAM, network permissions, human attention, and money across projects. It shall optimize the portfolio subject to hard safety and resource constraints rather than greedily running every project immediately.

A conceptual risk-sensitive objective is:

$$\min_{\pi} \quad \mathbb{E}\left[C_\pi + \lambda_D L_{\text{deadline}} + \lambda_F L_{\text{failure}} + \lambda_H H_{\text{human}} + \lambda_S R_{\text{security}} + \lambda_W W_{\text{waste}}\right] \tag{12.1}$$

$$+ \beta\, \text{CVaR}_\alpha\left(C_\pi + L_{\text{deadline}}\right), \tag{12.2}$$

subject to budgets, resource envelopes, safety invariants, worker compatibility, and salvage reserves. The conditional-value-at-risk term penalizes bad tail outcomes rather than optimizing only average cost.

12.2 Ordinary-user fairness

An ordinary user may have only one active project. A second submission is rejected until the first reaches a terminal state. The master administrator may operate multiple projects. When several ordinary projects are ready, the scheduler uses weighted fair service with deadline and dependency adjustments rather than permanent priority based only on submission order.

12.3 Calendar planning

The dashboard presents a calendar and Gantt-like schedule. Users may specify:

- desired deadline;
- maximum funded amount;
- economy, balanced, maximum-quality, or emergency-delivery mode;
- periods during which the computer should remain quiet;
- whether parallel execution is allowed;
- minimum acceptable completion probability.

The scheduler explains infeasible combinations rather than silently lowering quality.

12.4 Critical path

Task dependencies form a directed acyclic graph when possible. The system identifies the path that determines completion time and prioritizes work that unblocks other tasks. A small low-cost task with high blocking value may outrank a larger standalone feature.

12.5 Portfolio digital twin

The digital twin simulates many possible portfolio outcomes using distributions over:

- task duration and cost;
- model success and rework;
- test failure and reviewer rejection;
- human response time;
- worker-node availability;
- API rate limits and outages;
- resource contention.

Outputs include probabilities that projects finish by a deadline, probability of budget overrun, expected human decisions, most likely bottleneck, and marginal value of additional funding or faster response.

12.6 Value of information

The scheduler may authorize a low-cost proof task before a large implementation. For example, a $1 experiment that determines whether session recovery works may avoid a $15 failed architecture. Such tasks are assigned information value in addition to direct product value.

12.7 Operating schedule

The default POSM profile uses America/Chicago time:

- email window: 8:00 AM through 11:00 PM;
- maximum outbound supervisor emails: 10 per user per day;
- normal active compute window: 8:00 AM through 1:00 AM;
- quiet window: 1:00 AM through 8:00 AM.

Quiet mode permits already-approved long tests, batch analytics, indexing, and inexpensive telemetry processing. It does not begin expensive new milestones, Opus escalations, or tasks likely to require immediate human input unless an authorized overnight-sprint mode is active.

12.8 Machine-aware scheduling

Node selection accounts for CPU, RAM, disk, architecture, GPU, test intensity, current user activity, and network policy. A user-facing `I am using the computer` mode reduces concurrency and resource

envelopes. The governor always reserves enough resources for the host, database, workflow runtime, dashboard, and emergency shutdown.

Chapter 13

Sentinel: Statistical Intelligence, Security, and Improvement

13.1 Role

Sentinel is the fifth logical pair and has a different mission from project Cells. It audits and improves the system rather than owning a normal product project. Its deterministic telemetry service is continuously active; its statistical models update when data arrive; its LLM supervisor and Claude improvement worker wake only when an anomaly, scheduled review, or bounded experiment requires judgment or code.

13.2 Data integrity

Sentinel may transform, aggregate, and model telemetry, but it may never modify or delete raw events. The immutable event stream is the source of truth. Derived features, labels, posterior estimates, and recommendations are versioned and reproducible from raw event identifiers.

13.3 Responsibilities

Sentinel monitors:

- actual versus quoted cost and duration;
- probability of budget and deadline success;
- meaningful progress and stagnation;
- repeated worker failure patterns;
- supervisor efficiency and unnecessary turns;
- model-routing quality;
- contract drift and weak verification;
- dependency and secret risks;
- telemetry integrity and dashboard mismatch;
- resource use and scheduling fairness;
- opportunities for reusable skills and shared components.

13.4 Research-grounded model stack

Sentinel's probabilities shall come from registered statistical models, not LLM intuition.

13.4.1 Hierarchical Bayesian models

Partial pooling estimates cost and duration across repositories, languages, task types, models, and supervisors. New projects borrow strength from portfolio data while preserving project-specific uncertainty.

13.4.2 Survival analysis

Time-to-completion is represented as a distribution, enabling statements such as median completion time and 90 percent completion horizon rather than a single brittle estimate.

13.4.3 Hidden semi-Markov state model

Telemetry emissions are used to infer latent states such as exploring, implementing, verifying, progressing, reworking, stalled, blocked, or complete. Duration modeling helps distinguish normal exploration from abnormal stagnation.

13.4.4 Bayesian online changepoint detection

The Adams–MacKay framework estimates the posterior distribution over time since the latest regime change and is appropriate for detecting abrupt changes in burn rate, failure rate, or token efficiency [1].

13.4.5 Contextual bandits with knapsack constraints

Model routing can be framed as choosing an action under context while consuming finite money, time, compute, and human-attention budgets. Contextual bandits with knapsack constraints provide a relevant research foundation [2].

13.4.6 Safe Bayesian optimization

Review frequency, context size, routing thresholds, and concurrency may be tuned through bounded experiments that preserve explicit safety constraints. Safe Bayesian optimization research motivates cautious expansion from known safe configurations [10, 22].

13.4.7 Conformal calibration

Prediction intervals and risk estimates shall be empirically calibrated. Conformal risk control provides a basis for distribution-free or weak-assumption calibration of monotone loss, and anytime-valid extensions are relevant as Sentinel's calibration set grows [4, 16].

13.5 Model cards

Every production statistical model has:

- defined prediction target;
- feature specification;
- likelihood and prior rationale;
- training and validation periods;
- calibration metrics;
- subgroup and project-type performance;
- known weaknesses;
- rollback version;
- authority boundaries.

The dashboard shall display model version, sample size, interval coverage, Brier score or other relevant metrics, and a confidence class for every probability shown.

13.6 Earned autonomy

New Sentinel policies begin in historical replay, then shadow mode, then bounded experiments, and only later receive limited authority. A recommendation category may become autonomous after sufficient correct decisions and calibration. Database migrations, red permissions, safety-invariant changes, and budget increases remain human-controlled regardless of past accuracy.

13.7 Improvement laboratory

The Claude member of the Sentinel pair may develop improvements in an isolated repository. It creates pull requests; it cannot deploy or merge itself. An experiment contains a hypothesis, metric, budget, sample plan, stop rule, success criterion, safety constraints, and adoption decision.

13.8 Automatic pause authority

Sentinel may automatically pause or quarantine when it detects a hard budget violation, secret-like value, sandbox escape, cross-project access, repeated identical failure, severe dependency vulnerability, telemetry corruption, or an attempt to disable policy enforcement. It may not increase its own budget, alter safety invariants, approve red actions, erase evidence, or change another project's contract.

Chapter 14

Desktop Interface and Observability

14.1 Visual direction

The interface is a dark, information-dense mission-control extension of the POSM website aesthetic: near-black surfaces, restrained photonic blue and cyan accents, thin geometric borders, high-quality typography, subtle luminous gradients, and smooth event motion. It should feel scientific and premium rather than like a generic enterprise dashboard or gaming interface.

14.2 Progressive disclosure

The interface makes all useful information available without placing every raw field on the first screen. Three density modes are provided:

Executive status, decisions, budget, deadlines, and deliverables;
Engineering live prompts, commands, diffs, tests, and evidence;
Forensic complete tool events, token categories, policy decisions, hashes, and provenance.

The default for the master administrator is information-heavy Engineering mode.

14.3 Portfolio command center

The global header displays system health, active workers, daily and monthly spend, machine CPU and RAM, pending decisions, security alerts, and current operating mode. Global controls include New Project, Schedule, Decision Inbox, Economy Mode, Pause New Work, Stop All Workers, and Emergency Shutdown.

Each project card displays:

- current state, milestone, and task;
- supervisor and worker models;
- scope implemented and independently verified;
- milestone confidence and deadline probability;
- worker, supervisor, reviewer, and total spending;
- forecast range and remaining balance;

- tests and open failures;
- last meaningful progress;
- current worker action and blocker.

14.4 Live execution stream

The project Live view is the central trust feature. It shows events in real time:

```
10:42:11 SUPERVISOR Sent task TASK-019
10:42:14 CLAUDE Inspecting serializer implementation
10:42:16 FILE READ src/graph/serializer.ts
10:42:42 FILE EDIT +42 / -18 lines
10:42:56 COMMAND npm test -- serializer
10:43:04 TEST 18 passed, 2 failed
10:43:31 FILE EDIT graphResolver.ts
10:43:48 TEST 20 passed
10:43:51 SUPERVISOR Verifying REQ-GRAPH-014
```

Every event expands to show full safe input/output, parent prompt, cost delta, token usage, duration, process and container, Git state, permission rule, redaction status, and cryptographic provenance.

14.5 Project workspace tabs

The desktop workspace contains:

- Overview;
- Live;
- Contract;
- Roadmap and Schedule;
- Code and Diffs;
- Evidence;
- Costs;
- Decisions and Change Requests;
- Permissions and Security;
- Artifacts;
- History and Flight Recorder.

A three-column layout places navigation on the left, the main live or analytical view in the center, and a context inspector on the right.

14.6 Progress representation

The UI shall not display one unexplained completion percentage. It separately shows:

- scope with implementation evidence;
- scope independently verified;
- current milestone confidence;

- deadline success probability;
- budget consumed;
- progress velocity;
- time since meaningful progress.

Each metric is clickable and displays definition, formula, source events, timestamp, model version, and estimate/reconciliation status.

14.7 Decision inbox

The centralized Decision Inbox ranks items by urgency, critical-path delay, security level, and economic impact. Related questions are bundled. Each item presents options, recommendation, consequences, default action, deadline, and cost impact. The user may answer in the desktop application or by authorized email when policy allows.

14.8 Schedule and digital twin views

A calendar shows planned execution, quiet periods, milestones, reviews, and resource reservations. The digital twin view provides scenario controls for budget, deadline, concurrency, and response delay, with distributions rather than single deterministic outputs. Custom interactive visualizations are justified because they directly support resource and risk decisions.

14.9 Remote access

The desktop app remains the primary interface even when the backend runs in the lab. An authenticated private web endpoint may provide equivalent remote access. Public port forwarding is not required; private tailnet access is preferred.

Chapter 15

Repository, Artifacts, and Verification Architecture

15.1 Git integration

qFoundry shall use a dedicated GitHub App rather than a broad personal access token. GitHub App installation tokens can be limited to selected repositories and permissions and expire after a short period [11, 12].

A project may authorize:

- repository clone and read;
- feature-branch creation;
- commits and pushes to the feature branch;
- issue and pull-request creation;
- status and check reporting.

Protected-branch merge remains a two-step human action.

15.2 Worktrees and checkpoints

Every worker receives an isolated worktree or repository clone inside its container. Meaningful progress creates a checkpoint containing Git state, uncommitted diff if any, contract version, task state, session identifier, test summary, and cost ledger sequence. Rollback may target the last task, milestone, or named checkpoint.

15.3 Progressive verification

Verification runs from cheap to expensive:

1. syntax and static parsing;
2. changed-file lint and type checks;
3. targeted tests;
4. module suite;

5. full repository suite;
6. dependency and secret scans;
7. end-to-end acceptance tests;
8. independent milestone review.

The system shall not run the full expensive suite after every trivial edit unless the contract requires it.

15.4 Artifact-aware evidence

Evidence depends on project type.

15.4.1 Web applications

Screenshots, responsive layouts, browser console logs, network errors, accessibility scans, navigation crawl, and synthetic user journeys.

15.4.2 Numerical software

Reference cases, unit checks, error bounds, sensitivity analysis, independent implementation comparisons, and regression datasets.

15.4.3 EDA and PCB tools

Generated design files, DRC output, stackup assumptions, part models, manufacturing constraints, and reproducibility information.

15.4.4 FPGA systems

Simulation waveforms, synthesis report, timing closure, resource use, constraints, and hardware-test checklist.

15.5 Independent reviewer

Major milestones are reviewed by a fresh agent or human that receives the contract, diff, and evidence but not the directing supervisor's presumption of success. The reviewer looks for contract violations, missing edge cases, fake tests, architecture drift, security issues, numerical errors, and unnecessary complexity.

15.6 Golden scenarios

Permanent end-to-end tests include:

- create project, discover, approve contract, start worker, stream events, complete milestone;
- red permission, safe pause, email, authorized reply, secure confirmation, resume;
- budget threshold, salvage mode, additional funding, exact resume;
- crash during command or email, restart, no duplicate side effect;
- cross-project read attempt denied;

- false worker completion caught by verification.

15.7 Deliverable package

A completed project exports:

- executive and technical summary;
- contract and change history;
- repository branch, pull request, or archive;
- build artifact when applicable;
- tests and verification evidence;
- screenshots or reports;
- known limitations;
- cost report;
- reproduction instructions;
- security report;
- recommended next work.

Part III

Security, Safety, and Reliability

Chapter 16

Threat Model and Safety Invariants

16.1 Assets

qFoundry protects:

- provider API credentials and paid wallet balances;
- user repositories, source code, research data, and artifacts;
- project contracts, decisions, and private communications;
- host filesystem, user accounts, browser profiles, SSH keys, and other unrelated data;
- GitHub and Gmail integrations;
- immutable telemetry and financial ledger;
- physical compute hardware and laboratory network;
- the integrity of supervisor, policy, and Sentinel configuration.

16.2 Adversaries and failure sources

The threat model includes:

- a malicious or careless ordinary user;
- prompt injection embedded in repositories, issues, web pages, package metadata, or tool output;
- a compromised dependency or registry account;
- a coding agent that misunderstands instructions or executes a destructive command;
- a supervisor that approves an unsafe request;
- credential leakage through logs, environment variables, Git, email, traces, screenshots, or crash reports;
- cross-project data leakage;
- provider, network, database, or worker-node failure;
- manipulation or loss of telemetry used by Sentinel;
- unauthorized physical access to the lab machine;
- self-improvement code that attempts to weaken controls.

16.3 Immutable invariants

Security Invariant

1. No model receives raw long-lived credentials.
2. No worker may access a filesystem path outside its assigned sandbox.
3. No worker may access another project's repository, event stream, or artifact store.
4. No project may exceed its configured hard spending ceiling.
5. No AI may change its own permissions, budget, or safety invariants.
6. No AI may merge to a protected branch or deploy production code without the required human confirmation.
7. Raw security, financial, and telemetry events are append-only.
8. Every external side effect is attributable to actor, project, policy decision, tool call, and idempotency key.
9. Redacted secrets are never reconstructed for model display.
10. Emergency stop remains available independent of supervisor cooperation.

Invariants live in administrator-controlled configuration outside the contract and cannot be overridden through a normal change request.

16.4 Trust boundaries

The primary boundaries are:

- user client versus application API;
- application API versus orchestration services;
- orchestration services versus model gateway;
- credential broker versus all model-visible processes;
- worker container versus host and other containers;
- raw event ingestion versus derived Sentinel analytics;
- POSM deployment versus public local deployments.

16.5 Prompt injection policy

Repository and web content are data. Text that says "ignore prior instructions," requests secrets, tells the worker to disable tests, or claims to come from the user does not gain authority merely because Claude reads it. The execution pack explicitly states the authority hierarchy. High-risk untrusted input may tighten permissions mid-session.

16.6 Safety case

The security argument is defense in depth:

1. prevent unsafe requests through intake and policy;
2. constrain agents with least privilege and isolation;

3. detect anomalies through telemetry, scans, and Sentinel;
4. stop actions through hard budgets, permission gates, and kill controls;
5. recover through checkpoints, immutable records, and incident procedures;
6. prove boundaries through adversarial gate testing.

Chapter 17

Sandboxing and Resource Governance

17.1 Isolation requirement

A working directory is not a security boundary. A shell process may otherwise reach parent directories, user profiles, network shares, Docker sockets, or credentials. Each worker shall run inside an operating-system isolation boundary: initially a Linux container and, for higher assurance, optionally a lightweight virtual machine.

17.2 Filesystem

A standard worker sees:

```
/project writable project repository
/reference optional approved read-only material
/tmp ephemeral scratch space
/toolchain approved read-only tools
```

It shall not see the host home directory, browser profile, SSH directory, model gateway credentials, email tokens, other worktrees, or database files.

17.3 Container privilege

Workers run as non-root users without host PID, privileged mode, device access, or Docker socket. Rootless container operation reduces daemon and runtime privileges [9]. Container configuration uses a read-only root filesystem where compatible, dropped Linux capabilities, no-new-privileges, seccomp, and AppArmor or another mandatory-access-control profile.

17.4 Resource envelopes

Docker provides CPU and memory limits; without limits a container may consume host resources freely [8]. qFoundry configures:

- CPU quota and affinity;

- RAM hard and soft limits;
- process/PID maximum;
- disk and inode quotas;
- command wall-clock timeouts;
- maximum log volume;
- maximum open ports;
- maximum concurrent development servers;
- GPU assignment when applicable.

17.5 Per-user quotas

The POSM deployment supports configurable per-user CPU, RAM, disk, daily spend, project spend, runtime, process, and network limits. A user's one active project receives one resource envelope. The master administrator is exempt from the one-project rule but not from global host and budget constraints.

17.6 Network isolation

Default egress is denied. Approved destinations are proxied or allowlisted. Containers cannot connect to private laboratory devices or other project containers unless a contract-specific integration explicitly authorizes it. No inbound public port is created. Development servers bind to container or controlled loopback interfaces and are exposed through authenticated previews.

17.7 Worker images

Base images are versioned, scanned, reproducible, and minimized. Project-specific dependencies are layered above a stable base. Images include only required toolchains. Build caches are separated from credentials and can be invalidated without deleting project data.

17.8 Node trust

A worker node registers hardware capabilities and an attested qFoundry runner version. The scheduler does not place sensitive projects on an unknown or unhealthy node. Node compromise causes credential revocation and project quarantine.

17.9 Physical controls

The dedicated POSM machine is housed in McLeod's lab in a secured enclosure with approved airflow. Physical access is limited and documented. At least one backup administrator should be authorized for emergency shutdown and recovery even if normal access is reserved to the primary POSM administrator. The system uses a UPS, full-disk encryption, secure boot where practical, and automatic screen/session lock.

Chapter 18

Secrets, Credentials, and Authentication

18.1 Credential architecture

Long-lived secrets are stored in an encrypted vault controlled by a dedicated credential-broker service. The local root of trust should use an operating-system protected store or a well-reviewed encrypted application vault. For the Windows development deployment, Windows-protected credentials combined with Tauri Stronghold or an equivalent encrypted store are appropriate. The eventual Linux server should use a service secret store with encrypted disk and restricted service identities.

18.2 Operation proxying

Agents request operations rather than raw credentials. For example:

```
create_branch(project="posmq", branch="feature/serializer")
```

The broker validates project identity, permission, repository scope, and idempotency key; obtains an ephemeral token; performs the operation or passes the token only to a non-model integration process; then destroys the token.

18.3 Provider credentials

OpenAI and Anthropic keys exist only in the model gateway's environment or secret store. Claude worker shell commands and supervisor prompts never receive those values. Development and production deployments use separate provider projects and keys. Keys have provider-side spending limits where available, but qFoundry's internal governor remains the hard enforcement layer.

18.4 GitHub credentials

The system uses a GitHub App with minimum required permissions. Installation tokens are short-lived and restricted to approved repositories. The App private key remains in the broker. Workers do not receive a broad personal token.

18.5 Gmail credentials

The email service uses OAuth 2.0 server-side authorization and stores the refresh token in the encrypted vault. Gmail documents server-side authorization for applications that need access while the user is offline [14]. The email service is separate from worker containers.

18.6 Redaction pipeline

Before content enters logs, traces, screenshots, email, or model prompts, a redaction service scans for:

- known credential formats;
- high-entropy token-like strings;
- values matching secret-store fingerprints;
- private keys and certificates;
- authorization headers, cookies, and connection strings;
- project-defined sensitive patterns.

Redaction occurs at ingestion, not merely at display. A model or administrator can see that a secret was redacted and the incident identifier, but not the original value.

18.7 Leak response

If a secret-like value appears:

1. stop or quarantine the responsible worker;
2. prevent propagation into ordinary event views;
3. revoke or rotate the credential;
4. search repository, Git history, artifacts, traces, and emails;
5. record a security incident with redacted evidence;
6. notify the master administrator;
7. resume only after validation.

18.8 User authentication

The POSM deployment accepts approved UMN accounts. Successful institutional authentication is necessary but insufficient; the local account must also be approved and active. Sensitive administrative actions require recent re-authentication and may require a second factor or device confirmation.

18.9 Public local deployment

Independent users may choose their own vault and identity configuration. The installation wizard shall warn against placing keys in repository `.env` files or worker-visible environments and shall include a security self-test.

Chapter 19

Network, Dependencies, and Supply-Chain Policy

19.1 Approved network classes

Default POSM allowlists include:

- OpenAI and Anthropic APIs through the gateway;
- GitHub through the integration service;
- Gmail and Google OAuth through the email service;
- Tailscale or equivalent private access;
- approved package registries;
- official documentation domains;
- contract-approved project APIs.

Workers do not receive unrestricted web browsing by default. A new domain request is yellow or red depending on sensitivity and data transfer.

19.2 Package installation

Useful dependencies are encouraged when they reduce complexity, improve correctness, or provide maintained functionality. The policy is not "avoid packages"; it is "understand the dependency."

Before automatic approval, qFoundry checks:

- registry and normalized package name;
- publisher or ownership signals;
- exact version and lockfile change;
- license policy;
- known vulnerabilities;
- package age and maintenance activity where available;
- native binaries and install scripts;
- transitive dependency expansion;
- whether an existing approved dependency already provides the function.

The supervisor receives an audit event even for low-risk automatic approvals.

19.3 Dependency confusion and typosquatting

Package names proposed from model memory are verified against the registry. Private package namespaces are configured to prevent public-registry substitution. Similar-name and newly published packages receive heightened review.

19.4 Lockfiles and reproducibility

Every supported ecosystem uses a lockfile or equivalent pinned dependency manifest. Builds record base image, compiler, package-manager, registry, and dependency hashes. A project deliverable includes reproducibility instructions.

19.5 Web content as untrusted input

Official documentation may still contain code that is inappropriate for the project. Tool output cannot modify the authority hierarchy. A browser or documentation tool has read-only status unless a human or deterministic policy explicitly grants a separate action.

19.6 Network event visibility

The dashboard records destination category, domain, method, bytes, policy rule, and project. Sensitive request bodies are redacted. Sentinel detects new destination patterns, unusual volumes, or attempts to contact private network addresses.

Chapter 20

Acceptable Use, Abuse Handling, and Incident Response

20.1 Intake screening

Before execution, each project receives a lightweight but explicit safety review. The review evaluates stated purpose, repository content, requested integrations, likely external effects, financial source, and whether the project falls into a prohibited category.

20.2 Zero-tolerance intentional abuse

Intentional attempts to steal credentials, access another project, evade the sandbox, create malware, or disable controls trigger immediate account freeze and project quarantine. There is no graduated warning sequence for deliberate abuse.

Because a classifier or model can be wrong, qFoundry distinguishes immediate containment from final adjudication:

1. automatic freeze and evidence preservation;
2. master-administrator review;
3. confirmed abuse results in permanent ban;
4. institutional or external reporting occurs according to policy and legal obligation;
5. false positives are reversed and used to improve detection.

20.3 Incident severity

Level	Example	Response
SEV-0	active credential theft, cross-tenant compromise, destructive host access	global emergency stop, revoke credentials, isolate node, immediate administrator notification

SEV-1	secret exposure, sandbox escape attempt, malicious project	quarantine project and account, rotate affected secrets, preserve evidence
SEV-2	severe dependency vulnerability, telemetry corruption, repeated unsafe command	pause project, investigate, patch or rollback
SEV-3	ordinary policy error or suspicious but low-impact anomaly	supervisor/Sentinel review, no destructive action

20.4 Incident record

An incident record contains timeline, actors, projects, raw-event references, detection mechanism, containment actions, credentials affected, repository state, root cause, corrective actions, and closure approval. Raw secrets are never copied into the incident document.

20.5 Canary repository

The verification suite includes a repository containing fake keys, malicious instructions, dangerous scripts, misleading tests, dependency-confusion bait, and attempts to impersonate the user. Every release must demonstrate that these attacks are denied, contained, or escalated.

20.6 External reporting

The system shall not automatically contact law enforcement, providers, or University security based solely on an unreviewed model classification. Credible threats and legally required incidents are escalated to the appropriate human authority with preserved evidence.

Chapter 21

Data Retention, Privacy, and Provenance

21.1 Data classes

qFoundry classifies data as public, internal, project confidential, sensitive, or secret. The classification affects retention, model routing, screenshot capture, remote access, and Sentinel visibility.

21.2 Default retention

The initial POSM policy is:

Data	Default	Rationale
Contracts and change requests	project lifetime	authoritative product record
Decisions and permissions	project lifetime	audit and reproducibility
Cost ledger	two years	financial analysis and reconciliation
Security incidents	two years	incident history
Git and verification evidence	project lifetime	deliverable integrity
Full prompts and responses	30 days	forensic utility with bounded storage
Terminal and tool output	30 days	debugging and audit
Screenshots/recordings	14 days	high storage volume
Routine debug logs	14 days	operational troubleshooting
Raw secrets	never	prohibited by design

Users may export raw history before expiration. Deletion is requested from the master administrator so legal, security, and financial retention requirements can be checked.

21.3 Sensitive mode

A project may enable sensitive mode, reducing raw log retention, disabling screenshots by default, limiting Sentinel to telemetry unless investigation is approved, and restricting model/provider use according to deployment policy.

21.4 Sentinel learning

Sentinel may learn from aggregated cross-project telemetry by default. Source code is not a general training corpus. Sentinel obtains read-only project-scoped code access only for a documented security or progress investigation. Derived training data excludes secrets and respects project classifications.

21.5 Trace data

Agent traces can contain prompts, tool calls, and model output. OpenAI's Agents SDK tracing captures these event types and supports custom tracing processors [20]. qFoundry applies redaction and retention before exporting traces to any external observability backend.

21.6 Provenance

Every event records source service, collector version, timestamp, project, actor, parent event, policy decision, and hash chain. Displayed metrics preserve the identifiers of contributing events. Corrections create new events and do not rewrite prior data.

21.7 User transparency

The project settings show what is stored, where it is stored, when it expires, which providers receive content, whether Sentinel may access source, and which exports exist. Privacy controls are comprehensible without reading implementation code.

Chapter 22

Reliability, Recovery, and Operational Safety

22.1 Failure model

qFoundry anticipates provider outages, network loss, API rate limits, invalid credentials, exhausted billing, database failure, workflow-engine failure, worker crash, disk pressure, Git conflict, Gmail watch expiration, and full host restart.

22.2 Health states

Services report Healthy, Degraded, Unavailable, or Quarantined. The global dashboard distinguishes a provider outage from a project bug and a stale dashboard from stopped work.

22.3 Recovery objectives

The draft targets are:

- no loss of approved contracts, ledger entries, or permission decisions after acknowledgement;
- recovery of active project state after host restart;
- no duplicate external side effects after retry;
- automatic resumption of safe workflows when dependencies recover;
- explicit human review after security quarantine or ambiguous partial Git operations.

22.4 Backups

The system performs encrypted backups of database, contracts, ledger, workflow state, and critical artifacts. Active repositories remain in Git and are additionally included in checkpoint backup when unpushed changes exist. Backup media is separate from the main NVMe devices and periodically restored in a test environment.

22.5 Disk-pressure policy

At warning thresholds, qFoundry prunes build caches and expired raw logs. It does not delete active checkpoints, contracts, financial records, or unexported project artifacts. At a critical threshold, new projects and large builds pause.

22.6 UPS and shutdown

The dedicated lab workstation uses a UPS monitored by the host or independent watchdog. On extended power loss, qFoundry stops new calls, checkpoints workflows, flushes the database, interrupts workers, and shuts down cleanly.

22.7 Watchdog

A small independent device such as a Raspberry Pi may monitor server heartbeat, UPS state, temperature, and private network reachability. It can notify the administrator or request a graceful shutdown. It does not execute coding workloads or possess provider credentials.

22.8 Flight recorder

The flight recorder reconstructs prompts, commands, permissions, cost accumulation, Git state, resource use, email events, and supervisor decisions. It answers "why did this happen?" from evidence rather than model recollection.

Part IV

Deployment, Hardware, and Sustainability

Chapter 23

Desktop Client and Backend Deployment

23.1 Desktop-first product

The primary qFoundry experience is a polished Windows desktop application rather than a website tab. Tauri 2 supports cross-platform desktop applications using a web frontend and native Rust integration, with explicit application capability controls [23, 24]. The desktop application provides native notifications, system tray controls, secure local configuration, and a high-performance presentation layer for the live event stream.

23.2 Frontend stack

The recommended frontend stack is:

- Tauri 2 shell;
- React and TypeScript;
- a design-token system aligned with POSM branding;
- a component library used as a starting point rather than a visible template;
- Monaco for code and diff inspection;
- xterm.js or equivalent for terminal output;
- WebSocket or server-sent event transport for live telemetry;
- high-performance charting for the digital twin and cost data.

23.3 Local development deployment

The first implementation may run all services on a Windows development machine:

- Tauri desktop client;
- local API and orchestration services;
- PostgreSQL;
- Temporal development or self-hosted service;
- Docker Desktop with WSL2 Linux workers.

Microsoft documents Docker development with WSL2-backed Linux containers on Windows [17]. This

arrangement supports rapid development but is not the final multi-user lab security boundary.

23.4 Laboratory deployment

The eventual shared POSM backend runs on a dedicated Linux workstation in McLeod's lab. Desktop clients authenticate to the server through a private network overlay. The backend remains reachable when the user's laptop is closed, while repositories and secrets remain on controlled infrastructure.

23.5 Remote access

The server shall not expose an unauthenticated public port. A private tailnet or institution-approved VPN provides authenticated access. Tailscale Serve is one implementation option for making a local service available to authorized tailnet devices without public exposure.

23.6 Backend services

Recommended services include:

- API gateway and authorization;
- project and contract service;
- durable workflow workers;
- model gateway;
- ledger and reservation service;
- scheduler;
- credential broker;
- worker-node controller;
- event ingestion and streaming;
- Sentinel feature and model services;
- email and GitHub integration;
- artifact store.

23.7 Database

PostgreSQL stores relational state, fixed-point financial values, contracts, users, roles, project membership, permissions, and event metadata. Row-level or equivalent application authorization shall enforce tenant boundaries. High-volume event payloads may be partitioned or stored in an append-only object/event system with database indexes.

23.8 Packaging

The public release shall include:

- Windows installer;
- documented Linux server deployment;

- local single-user profile;
- managed multi-user profile;
- migration and backup tools;
- security self-test;
- provider-key setup wizard;
- sample project and canary repository.

Chapter 24

Dedicated Laboratory Compute Infrastructure

24.1 Compute characteristics

Cloud providers perform the large-model inference. The qFoundry server primarily runs repository operations, builds, tests, browsers, databases, event processing, containers, statistical analysis, and optional local models. Therefore the priority order is:

RAM > CPU cores > fast/high-endurance NVMe > reliability > network > optional GPU.

24.2 Initial machine objective

The dedicated computer's exclusive purpose is qFoundry operation. It should support four logical development Cells, Sentinel, databases, and the desktop backend, while initially limiting actual active Claude workers to two. Capacity can increase after measurement.

24.3 Preferred flagship configuration

A strong lab target is:

- 24-core/48-thread AMD Threadripper-class CPU or a donated previous-generation workstation equivalent;
- TRX50 or professional workstation platform;
- 128 GB ECC RAM minimum, 256 GB preferred;
- separate system, active-work, and artifact/cache NVMe devices;
- independent backup storage;
- 2.5 GbE minimum, 10 GbE preferred;
- high-quality PSU and monitored UPS;
- optional 16–24 GB VRAM GPU for local models and vision;
- strong cooling and filtered secured enclosure.

24.4 Attainable fallback

A Ryzen 9 9950X-class system with 128 GB RAM and two NVMe devices is a practical starting point and likely supports two to four ordinary workers depending on repository workload. The architecture should not delay software development until a perfect Threadripper donation appears.

24.5 Heterogeneous node pool

The most cost-effective expansion is a mixed pool of x86 workstations rather than a Raspberry Pi cluster. Retired Dell Precision, HP Z, Lenovo ThinkStation, or previous-generation Threadripper systems can each run one or more worker containers. The primary node hosts databases and control services; secondary nodes provide isolated execution capacity.

24.6 Raspberry Pi role

A Raspberry Pi is appropriate as an independent watchdog, UPS monitor, status display, Wake-on-LAN controller, or telemetry heartbeat. It is not the primary coding node because ARM compatibility, low per-node RAM, and cluster-management overhead outweigh the benefit for general software builds.

24.7 Environmental and physical requirements

Before installation, POSM confirms:

- laboratory approval and ownership route;
- continuous power and appropriate circuit capacity;
- wired network and private remote access;
- acceptable heat and noise;
- airflow through the secured case;
- UPS placement and shutdown signaling;
- backup administrator and emergency access;
- asset inventory and insurance requirements.

24.8 Resource monitoring

The dashboard exposes CPU, RAM, disk, temperature where available, network, process count, and worker allocation. Sentinel uses telemetry to forecast capacity and detect resource leaks. The scheduler reserves capacity for the operating system and emergency services.

Chapter 25

Hardware Procurement, Donations, and Sponsorship

25.1 Funding constraint

POSM is willing to contribute up to approximately $1000 toward the first dedicated machine. The procurement strategy therefore combines direct purchase of reliability-critical parts with component donations, academic programs, surplus, refurbishment, and discounts.

25.2 Parts POSM should buy

POSM should preferentially buy new:

- primary power supply;
- boot/service NVMe;
- required CPU cooler and thermal materials;
- cables and small compatibility items;
- new or verified UPS batteries;
- at least one known-good backup device.

These components are affordable relative to the total system and have disproportionate reliability impact. The exact motherboard and RAM should not be purchased before the CPU/platform is fixed.

25.3 High-value donation targets

The strongest formal route is the AMD University Program, which accepts academic hardware and software donation requests for teaching and research [3]. A faculty or senior staff sponsor should submit or endorse the request.

Other component-specific targets include:

Target	Requested subsystem	Best framing

AMD	CPU, GPU, mini-PC, cluster access	academic AI engineering and scientific-software infrastructure
Micro Center	store credit, open-box motherboard/CPU, assembly, matching discount	local student engineering platform with visible impact
DigiKey	watchdog, sensors, power monitoring, SBC, cabling, product credit	independent qFoundry safety and infrastructure subsystem
Seagate / WD / Solidigm / Kingston	NVMe, enterprise backup, ECC memory	telemetry, build-cache, checkpoint, and storage sponsorship
Noctua / Fractal / Seasonic	cooling, case, PSU	clearly bounded official subsystem sponsorship
APC / Eaton / CyberPower	UPS chassis or academic discount	reliable autonomous workflow and graceful shutdown
Dell / HP / Lenovo / local employers	retired engineering workstations	secondary worker nodes after secure wipe
UMN ReUse	monitors, network gear, racks, secondary systems	campus surplus reuse

DigiKey operates an academic program for colleges and universities and invites project-specific contact [7]. UMN ReUse collects surplus equipment from across campus and makes usable items available to departments or the public [27].

25.4 Donation probability philosophy

The specification does not treat informal probability estimates as published acceptance rates. Planning ranges should be updated from actual outreach outcomes. A mixed campaign has better portfolio probability than a single all-or-nothing request.

25.5 Legitimate leverage strategies

Requests should explicitly accept:

- previous-generation hardware;
- evaluation or demonstration units;
- recertified or open-box inventory;
- cosmetically damaged packaging;
- retired internal engineering workstations;
- academic discounts, store credit, or matched contributions;
- compute or API credits when hardware is unavailable.

Each company should receive a specific subsystem request rather than a generic full shopping list.

25.6 Sponsor packet

The packet includes:

1. one-page overview;
2. exact component and acceptable alternatives;
3. architecture diagram showing its role;
4. student and research impact;
5. sponsor recognition and technical case-study plan;
6. POSM legitimacy and faculty support;
7. ownership, location, maintenance, and no-resale plan;
8. fulfillment and impact report.

25.7 Ownership route

Before accepting equipment, POSM determines whether the item is donated directly to POSM, donated to the University for dedicated POSM use, or loaned. Donor-facing tax and ownership language must match the approved route. Lab storage and access are documented in writing.

Chapter 26

Open-Source Release, Branding, and Sustainability

26.1 Release intent

After a private validation period, qFoundry is intended to be free software that other individuals and laboratories can install, configure with their own model credentials, and run locally. POSM branding remains visible, while deployment policies are configurable.

26.2 License direction

A provisional recommendation is AGPLv3 for the server and orchestration core, with compatible licenses for client components. AGPL encourages operators who modify and provide the system over a network to publish corresponding source changes. A final decision requires legal review.

A noncommercial-use restriction would make the license source-available rather than open source under the conventional Open Source Definition. Therefore POSM should separate:

- software freedom and code license;
- POSM-hosted-service acceptable-use rules;
- POSM name and logo trademark policy;
- optional commercial support or dual-licensing decisions.

26.3 Configuration profiles

Public users can choose:

Local personal one user, own keys, local storage, user-defined budgets;
Lab managed multiple users, administrator, wallet, quotas, shared workers;
Developer mock providers, canary repositories, low test credits;
High-security stricter isolation, limited integrations, reduced retention.

26.4 No mandatory POSM caps

The $30 project cap, UMN-only accounts, one-project-per-user rule, email window, and POSM safety categories are configuration for the POSM service. The open-source engine exposes policy primitives and sensible defaults but allows a local operator to change them.

26.5 Project governance

Public governance should include:

- security policy and private vulnerability reporting;
- contribution guidelines;
- architecture decision records;
- versioned contract and event schemas;
- migration policy;
- release signing;
- maintainer code of conduct;
- transparent compatibility matrix for providers and worker environments.

26.6 Economic sustainability

POSM's installation may be centrally funded, user funded, sponsor funded, or mixed. Public users supply their own keys by default. The software shall not require POSM to subsidize public usage. Future optional services could include managed hosting, institutional support, or certified deployment without limiting self-hosting.

Part V

Verification and Delivery

Chapter 27

Verification Gates

27.1 Gate philosophy

No subsystem is trusted because documentation says it exists. qFoundry advances through proof-of-capability gates with small API credits, canary repositories, forced failures, and measurable acceptance criteria.

27.2 Gate 1: Claude control spike

Demonstrate programmatic start, streaming, tool events, permission request, approval and denial, interrupt, session continuation, usage retrieval, file inspection, and state restoration.

27.3 Gate 2: closed-loop supervision

One supervisor directs Claude through at least five consecutive cycles without human prompting. The sequence includes implementation, verification failure, corrective prompt, successful retest, and milestone advancement.

27.4 Gate 3: observability parity

The desktop stream shows prompts, responses, tool calls, commands, output, file operations, tests, permissions, errors, cost, and process health with bounded latency and no unexplained gaps.

27.5 Gate 4: contract compiler experiment

Two comparable workers receive either a raw contract or compiled execution pack. Compare requirements missed, tokens, questions, rework, cost, and acceptance success. The compiler must demonstrate value before becoming mandatory.

27.6 Gate 5: budget hard stop

Use a tiny supervisor and worker budget. Confirm thresholds, reservation, no unsafe overrun, salvage report, session resume after additional funding, and reconciliation.

27.7 Gate 6: sandbox and secret red team

Attempt to read host files, another project, fake SSH keys, environment credentials, private network destinations, malicious README instructions, and suspicious packages. All attempts must be denied, quarantined, or escalated.

27.8 Gate 7: crash and idempotency

Crash during a worker response, file write, test, Git operation, email send, ledger update, and human wait. Confirm state recovery and no duplicate side effects.

27.9 Gate 8: email loop

Send a signed escalation during the communication window, ingest an authorized reply, attach it to the correct project, interpret it, require secure confirmation where appropriate, resume exactly once, enforce the ten-email daily maximum, and queue out-of-window mail.

27.10 Gate 9: time–cost planning

Give one project three deadline/budget scenarios. Confirm different task order, model routing, batching, concurrency, confidence, and critical path. Introduce delays and verify rescheduling.

27.11 Gate 10: supervisor calibration

Run a scenario library of dependencies, architecture ambiguity, secrets, low-value expensive work, false completion, and contract conflict. Compare supervisor recommendations with human ground truth and resulting outcomes.

27.12 Gate 11: Sentinel anomaly detection

Inject cost overrun, repeated failure, stagnation, secret string, contract drift, telemetry mismatch, and self-reverting code. Measure detection delay, false positives, and intervention quality.

27.13 Gate 12: multi-project load

Run five logical projects with limited actual concurrency. Verify budget and data isolation, CPU/RAM limits, rate-limit response, dashboard performance, schedule fairness, and restart recovery.

27.14 Gate 13: public local install

A clean external machine installs qFoundry, supplies its own keys, creates a project, runs a bounded task, and uninstalls or exports data without relying on POSM infrastructure.

Chapter 28

Acceptance Criteria and Quality Metrics

28.1 Version 0.1 release criteria

The first release is accepted only when:

1. discovery produces a complete versioned contract;
2. Claude completes five autonomous supervisor-directed cycles;
3. the supervisor catches an intentionally incomplete claim;
4. the live stream exposes all material actions and evidence;
5. a tiny hard budget stops safely;
6. fake secrets never appear in model-visible logs, Git, email, or screenshots;
7. one worker cannot access another repository or protected host path;
8. email reply resumes the correct project once;
9. restart recovery preserves state and avoids duplicate effects;
10. the operator can pause, take over, and return control;
11. two projects compete for limited resources correctly;
12. Sentinel detects seeded cost, security, stagnation, and contract anomalies;
13. one real bounded project is independently verified;
14. cost estimates and reconciled usage are clearly distinguished;
15. every dashboard claim links to evidence.

28.2 Performance targets

Initial targets, subject to measurement:

- live event display latency below two seconds for 95 percent of ordinary events on the local network;
- project-state recovery below two minutes after application restart excluding long worker rebuilds;
- decision and ledger writes durably acknowledged before UI success;
- no project event visible to an unauthorized user in automated isolation testing;
- no request initiated when conservative reservation exceeds remaining hard budget.

28.3 Statistical quality

Sentinel models are evaluated using appropriate metrics:

- interval coverage and width for cost and duration;
- Brier score, log loss, calibration curve, and expected calibration error for probabilities;
- detection delay and false-positive rate for changepoints/anomalies;
- regret and constraint violations for routing experiments;
- task success, cost, time, human burden, and reversion rate for policy comparison.

28.4 Supervisor quality

Track:

- tasks completed within quote;
- unnecessary worker turns;
- incorrect approvals;
- contract violations caught;
- false completions accepted;
- escalations later judged unnecessary;
- agreement between recommendation and human decision;
- cost per accepted requirement and milestone;
- reverted or abandoned work.

28.5 Usability quality

Users should understand current task, spend, blocker, decision, and risk without reading raw logs. The forensic details remain accessible. The interface is tested for keyboard navigation, text scaling, contrast, and large event-stream performance.

Chapter 29

Implementation Roadmap

29.1 Phase 0: specification and threat model

Approve this document, resolve marked TBD items, finalize schemas, create architecture decision records, and establish the canary repository.

29.2 Phase 1: capability spike

Build:

- minimal Tauri desktop shell;
- one local project;
- one OpenAI supervisor;
- one Claude worker;
- streamed event normalization;
- one isolated worktree/container;
- one budget and reservation path;
- one permission request;
- pause/interrupt;
- basic test evidence.

The phase ends only when Gates 1–6 pass with low credits.

29.3 Phase 2: durable single-project system

Add contract discovery, compiler, Temporal workflow, crash recovery, checkpoints, Gmail escalation, GitHub App, full live UI, and acceptance evidence.

29.4 Phase 3: two-project portfolio

Add scheduler, resource envelopes, ordinary/test user separation, shared wallet scaffolding, decision inbox, and load testing with two workers.

29.5 Phase 4: four development Cells

Enable five logical slots, four normal project Cells, master portfolio view, user quotas, dedicated lab deployment, and hardware telemetry.

29.6 Phase 5: Sentinel

Begin with deterministic telemetry, registered baseline models, historical replay, and advisory reports. Add shadow routing, anomaly-triggered LLM review, improvement experiments, and the portfolio digital twin.

29.7 Phase 6: private POSM pilot

Operate for several months with approved UMN users, low initial caps, incident review, cost calibration, and regular architecture updates.

29.8 Phase 7: public preparation

Separate POSM policy from core configuration, harden installation, publish security documentation, select license, create migration tools, and perform external review.

29.9 Dependency principle

Do not build full multi-user payment, mobile apps, Kubernetes, public SaaS, or autonomous deployment before the core loop is safe and economical. The architecture may reserve interfaces for them without implementing them prematurely.

Chapter 30

Operational Runbooks

30.1 Daily startup

The administrator checks server health, backup status, provider balance, pending security alerts, expired Gmail watches, queued projects, UPS state, and available disk. qFoundry performs automated self-checks before admitting new work.

30.2 Project admission

Confirm approved user, one-project rule, contract approval, intake result, funded amount, repository permission, resource quote, and scheduled start. Admission creates the immutable project baseline event.

30.3 Budget stop

When salvage activates, confirm no new calls are admitted, current work is checkpointed, final verification remains within reserve, report is delivered, and the user has clear options.

30.4 Secret incident

Freeze worker, revoke credential, quarantine repository, search propagation surfaces, preserve redacted evidence, notify administrator, rotate, retest, and document closure.

30.5 Worker stagnation

Review meaningful-progress events, repeated commands, test trajectory, diff reversions, and quote assumptions. Choose diagnosis prompt, fresh session, model escalation, proof task, rollback, or human question. Do not continue identical retries.

30.6 Host outage

On restart, validate database and event-store integrity, restore workflows, reconcile pending reservations, identify orphaned containers, verify Git state, and resume only safe projects.

30.7 Provider outage

Pause affected calls, retain reservations appropriately, estimate schedule impact, use fallback only where approved, and notify users only when critical path or decision timing is affected.

30.8 Account abuse

Freeze account and projects, revoke credentials, preserve evidence, notify master, perform review, permanently suspend confirmed deliberate abuse, and record any external report.

30.9 Hardware maintenance

Schedule operating-system and container updates, test worker images, verify UPS batteries, check drive health, clean filters, review temperatures, and perform a restoration test. Maintenance windows are visible in the schedule.

Chapter 31

Risks, Limitations, and Open Decisions

31.1 Technical risks

- provider SDK behavior may change;
- exact live cost can lag provider reconciliation;
- container isolation on a Windows development host is weaker and more complex than a dedicated Linux server;
- coding agents may produce plausible but incorrect tests;
- long-running sessions may compact or lose subtle context;
- email interpretation can be ambiguous;
- a dense UI can overwhelm ordinary users;
- local statistical models will initially have little data;
- multi-node scheduling introduces operational complexity.

31.2 Organizational risks

- recurring API costs may exceed donated hardware value;
- system operation may depend too heavily on one administrator;
- faculty/lab ownership and access could be unclear;
- users may treat the service as a substitute for understanding their own projects;
- sponsorship commitments may imply reporting work;
- public release may create support burden.

31.3 Statistical limitations

Early Bayesian estimates use conservative priors and wide intervals. No model should claim precise completion probabilities with a tiny sample. Changes in provider models, pricing, repository types, or supervisor prompts create distribution shift. Calibration is monitored continuously, and uncertainty is shown prominently.

31.4 Open decisions

Before implementation baseline approval, POSM should finalize:

- exact source-code license and trademark policy;
- dedicated supervisor email account;
- final institutional ownership route for donated hardware;
- faculty/lab support letter;
- exact initial provider models and pricing tables;
- detailed database and event-store technology;
- initial secret-store implementation;
- first bounded pilot repository;
- final UI design tokens and logo treatment.

31.5 Review cadence

This specification is reviewed after the capability spike, after the first real project, after multi-user activation, and before public release. Change requests preserve why each major deviation occurred.

Part VI

Normative Appendices

Chapter A

Normative Requirement and Contract Schema

A.1 Purpose

This appendix defines the minimum structured representation for a qFoundry contract. The implementation may add fields but shall not remove semantics required for traceability.

```yaml
contract:
  id: QF-PROJECT-0001
  version: 1.0.0
  status: approved
  owner_user_id: user_123
  approved_at: 2026-06-14T21:00:00-05:00
  parent_version: null
  compiled_hash: sha256:...

product:
  name: Example Project
  vision: >
    Human-readable product objective.
  intended_users:
    - researcher
  non_goals:
    - production deployment
  glossary:
    milestone: A verified collection of requirements.

requirements:
  - id: REQ-LIVE-001
    title: Live execution stream
    priority: must
    rationale: >
      The operator must see worker progress and evidence.
    behavior:
```

```
      trigger: worker event accepted
      expected_result: event appears in project stream
      max_latency_ms: 2000
    dependencies: []
    owner_milestone: M-004
    acceptance:
      methods:
        - automated
      test_ids:
        - E2E-LIVE-001
    evidence_required:
      - event_record
      - ui_timestamp
    status: approved

nonfunctional_requirements:
  security: []
  reliability: []
  performance: []
  privacy: []
  accessibility: []

budgets:
  target_usd: "20.00"
  tolerance_usd: "0.50"
  hard_ceiling_usd: "20.50"
  salvage_reserve_percent: 5

permissions:
  policy_version: PERM-001

model_routing:
  default_worker_tier: sonnet
  opus_triggers:
    - repeated_diagnosed_failure
    - security_critical_task

milestones:
  - id: M-001
    title: Capability spike
    requirements:
      - REQ-LIVE-001
    dependencies: []
    definition_of_done:
      - all_must_requirements_verified

human_reserved_decisions:
  - production_deployment
  - hard_budget_increase
```

```
  - protected_branch_merge

change_control:
  human_approval_required: true
  active_change_requests: []
```

A.2 Requirement statuses

Status	Meaning
Draft	still under discovery
Approved	part of the current contract
In implementation	assigned to an active task
Implemented	code/artifact evidence exists but verification incomplete
Verified	required evidence passed
Waived	user-approved removal through change request
Blocked	cannot proceed because a named dependency or decision is unresolved
Failed	implementation or verification does not satisfy requirement

A.3 Change request schema

```
change_request:
  id: CR-0014
  project_id: example
  requested_by: user_123
  summary: Replace local queue with Temporal workflow
  reason: Restart recovery is required
  affected_requirements:
    - REQ-REL-004
  architecture_effects:
    - add temporal service
  cost_estimate_usd:
    p50: "2.40"
    p90: "4.80"
  schedule_effect_hours:
    p50: 5.0
  security_effect: medium
  supervisor_recommendation: approve
  human_decision: pending
  resulting_contract_version: null
```

A.4 Compiler outputs

The contract compiler emits a deterministic bundle manifest containing hashes of the human document, machine schema, task packs, acceptance plan, and requirement graph. A worker prompt references the

bundle and selected requirement identifiers.

Chapter B

Event and Telemetry Schema

B.1 Canonical event

```
{
  "event_id": "evt_01JXYZ",
  "schema_version": "1.0",
  "sequence": 18542,
  "occurred_at": "2026-06-14T22:43:48.123-05:00",
  "recorded_at": "2026-06-14T22:43:48.160-05:00",
  "portfolio_id": "posm",
  "tenant_id": "umn-user-tenant",
  "project_id": "posmq",
  "workflow_id": "wf_004",
  "session_id": "claude_018",
  "trace_id": "trace_281",
  "parent_event_id": "evt_01JXYA",
  "actor": {
    "type": "worker",
    "id": "claude-sonnet-A"
  },
  "event_type": "test.completed",
  "severity": "info",
  "payload": {
    "command": "npm test -- serializer",
    "exit_code": 0,
    "passed": 20,
    "failed": 0,
    "duration_ms": 7401
  },
  "cost": {
    "provider": "anthropic",
    "model": "sonnet",
    "input_tokens": 0,
    "cached_input_tokens": 0,
    "output_tokens": 0,
    "estimated_delta_micro_usd": 0,
```

```
    "reconciled_delta_micro_usd": null,
    "pricing_version": "anthropic-2026-06"
  },
  "security": {
    "classification": "internal",
    "redaction_applied": false,
    "policy_decision_id": "perm_118"
  },
  "provenance": {
    "source": "container-agent-a",
    "collector_version": "0.1.0",
    "previous_event_hash": "abc...",
    "event_hash": "def..."
  }
}
```

B.2 Core event taxonomy

Event type	Meaning
`project.created`	project shell and tenant boundary created
`contract.version.approved`	human approved a contract version
`quote.generated`	preflight plan created
`workflow.state.changed`	durable project state transition
`supervisor.prompt.sent`	task or correction sent to worker
`worker.message.partial`	streamed visible response chunk
`worker.message.completed`	worker turn completed
`worker.tool.requested`	worker proposed a tool operation
`permission.approved` / `permission.denied`	policy decision
`command.started` / `command.output` / `command.completed`	shell execution
`file.read` / `file.modified` / `file.deleted`	file activity
`test.started` / `test.completed`	verification result
`git.commit.created` / `pull_request.opened`	repository integration
`budget.reserved` / `budget.reconciled`	financial accounting
`budget.threshold.crossed`	threshold action
`human.escalation.created` / `human.response.received`	human-in-loop communication

`security.anomaly.detected`	security detector finding
`sentinel.posterior.updated`	statistical state update
`project.paused` / `project.resumed`	execution control
`artifact.created`	deliverable or evidence stored

B.3 Telemetry feature examples

Derived features include tokens per verified requirement, time since meaningful progress, test-pass trajectory, file reversion rate, unique commands, repeated-failure count, human-response delay, cache-hit ratio, cost-estimate error, and worker state probabilities. Each feature stores its source event range and transformation version.

B.4 Hash-chain limitations

A hash chain provides tamper evidence, not an absolute guarantee against a privileged administrator rewriting both data and hashes. Higher assurance may periodically anchor batch roots in a separate append-only backup or signed audit file.

Chapter C

Preflight Quote and Schedule Schema

C.1 Example quote

```yaml
quote:
  id: QUOTE-000219
  project_id: posmq
  contract_version: 1.2.0
  created_at: 2026-06-14T21:00:00-05:00
  expires_at: 2026-06-15T09:00:00-05:00

request:
  task_id: TASK-043
  desired_deadline: 2026-06-17T18:00:00-05:00
  maximum_budget_usd: "8.00"
  minimum_success_probability: 0.85
  quality_mode: balanced
  parallel_execution_allowed: false

repository_assessment:
  indexed: true
  languages: [TypeScript]
  complexity_class: medium
  baseline_test_health: partial
  uncertainty_sources:
    - incomplete integration tests

plans:
  - id: cheapest
    cost_usd: {p10: "2.10", p50: "3.40", p90: "6.20"}
    duration_hours: {p10: 1.5, p50: 3.0, p90: 7.0}
    deadline_probability: 0.81
    success_probability: 0.84
    worker_tier: sonnet
    supervisor_tier: standard_reasoning
    concurrency: 1

```

```yaml
  - id: recommended
    cost_usd: {p10: "3.20", p50: "4.80", p90: "7.10"}
    duration_hours: {p10: 1.0, p50: 2.2, p90: 4.5}
    deadline_probability: 0.93
    success_probability: 0.91
    worker_tier: sonnet
    opus_escalation: conditional

selected_plan: recommended

cost_reservation:
  worker_micro_usd: 1200000
  supervisor_micro_usd: 300000
  salvage_micro_usd: 800000

critical_path:
  - repository_map
  - implementation
  - integration_tests
  - review

risks:
  - id: RISK-001
    probability: 0.35
    impact: medium
    mitigation: proof_task_first

model_provenance:
  estimator_id: cost-duration-hierarchical-v0.3
  calibration_class: moderate
  comparable_tasks: 17
```

C.2 Quote validity

A quote expires when its deadline passes, repository baseline changes materially, provider pricing changes, contract version changes, a dependency fails, or a significant new observation updates the estimator.

C.3 Infeasibility result

An infeasible request returns the closest alternatives. Example: increase budget, relax deadline, reduce scope, permit parallelism, accept lower confidence, or run a proof task. The system shall not silently force an infeasible plan into a misleading estimate.

Chapter D

Permission and Authority Matrices

D.1 Core matrix

Actor	Action	Resource	Default	Condition
Claude	read/edit	assigned repository	green	mounted project path only
Claude	run tests/build	container	green	approved command family
Claude	install trusted package	approved registry	green/yellow	verified package, pinned version, policy score
Claude	delete file	repository	yellow	limited count and checkpoint
Claude	delete directory	repository	red	human approval
Claude	access other repository	filesystem	deny	invariant
Claude	access credential	vault	deny	operation proxy only
Claude	new network domain	network	yellow/red	purpose and data sensitivity
Claude	push feature branch	GitHub	conditional green	user pre-authorized
Claude	open pull request	GitHub	conditional green	user pre-authorized
Any AI	merge protected branch	GitHub	deny/red	two-step user confirmation outside worker
Supervisor	choose implementation detail	contract scope	green	no product change
Supervisor	approve yellow request	project	yellow	within delegated class
Supervisor	change contract	contract	deny	human change request

Supervisor	increase hard budget	ledger	deny	user/master approval
Sentinel	pause worker	project	green	registered anomaly rule
Sentinel	quarantine	project/node	green	security threshold
Sentinel	modify production	qFoundry code	deny	PR and human review only
User	approve own product contract	project	green	authenticated project owner
User	red security permission	project	deny	master required
Master	emergency stop	portfolio	green	always available

D.2 Green, yellow, red definitions

Green deterministic automatic approval under explicit policy;
Yellow supervisor or project owner may approve within delegated scope;
Red master or secure human confirmation required;
Deny no normal override; changing the rule requires administrator policy review.

D.3 Permission decision record

Every decision records proposed operation, normalized arguments, actor, project, matched rule, data classification, risk score, decision maker, expiry, and result. Natural-language summaries are secondary to normalized tool arguments.

Chapter E

Budget and Cost Examples

E.1 Project funding example

A user funds $20.00 and selects $0.50 tolerance. The absolute project ceiling is $20.50. The policy reserves five percent of the funded amount for salvage, but the user may choose a larger reserve.

Funded amount	$20.00
Positive tolerance	$0.50
Absolute ceiling	$20.50
Reconciled spend	$18.72
Pending estimates	$0.18
Salvage reserve	$0.60
Safely available	$1.00
Worst-case next call reservation	$1.24
Decision	Block and salvage

E.2 Wallet example

A user deposits or is allocated $60. The user funds the first project with $20. Only actual spend is consumed; unused funded value returns to the user's available balance after closure. Future payment implementation shall distinguish cash balance, promotional credit, institutional grant, and nonrefundable provider credit.

E.3 Sub-budget example

OpenAI supervisor	$3.00
Claude worker	$14.00
Independent reviewer	$1.00
Sentinel allocation	$0.50
Tools/compute reserve	$0.50

Salvage reserve	$1.00
Total	$20.00

Sub-budgets may borrow within the same user's funded project only under configured policy; they never allow the total ceiling to be exceeded.

E.4 Estimate calibration

Suppose 100 tasks receive nominal 90 percent cost intervals. Sentinel should expect approximately 90 realized costs inside those intervals, subject to calibration uncertainty. If coverage falls materially below target, the system widens intervals or reduces displayed confidence rather than continuing to advertise false precision.

Chapter F

Email Escalation and Reply Protocol

F.1 Addressing

The POSM deployment uses a dedicated qFoundry mailbox, with a placeholder address until created. Display names identify the project or Sentinel without impersonating OpenAI or Anthropic.

F.2 Communication policy

Timezone is America/Chicago. Ordinary outbound email is sent only between 8:00 AM and 11:00 PM, with a maximum of ten messages per user per calendar day. Nonurgent items are bundled. Security containment occurs immediately even if the corresponding notification is queued.

F.3 Escalation record

```yaml
escalation:
  id: ESC-0147
  project_id: eom-designer
  severity: blocking
  subject: Decision required: resonance topology
  question: Choose switched bank or variable inductor
  options:
    - id: A
      summary: Switched fixed-inductor bank
      cost_effect_usd: "0.80"
    - id: B
      summary: Variable-inductor architecture
      cost_effect_usd: "2.40"
  recommendation: A
  response_deadline: 2026-06-15T15:00:00-05:00
  signed_thread_token: opaque-token
```

F.4 Reply ingestion

The email service verifies sender allowlist, thread, signed token, freshness, and duplicate message identifier. Gmail push notifications can notify the backend of mailbox changes, and the message is then retrieved and attached to the project [13].

F.5 Interpretation and confirmation

The supervisor records a normalized interpretation. Ordinary decisions may resume automatically. Red actions display the interpreted operation in the desktop application and require secure confirmation. A reply such as “yes” is never allowed to approve an unspecified destructive action.

F.6 Rate limiting and bundling

Related questions are combined into one decision bundle. The system tracks daily count, severity, quiet hours, snooze, and dashboard-only preference. Duplicate provider or workflow retries do not generate duplicate mail.

Chapter G

Hardware Bill of Materials and Sponsorship Matrix

G.1 Flagship target

Subsystem	Preferred target	Fallback	Funding route
CPU	Threadripper 9960X class	Ryzen 9 9950X or prior-gen Threadripper	AMD donation or POSM purchase fallback
Motherboard	TRX50 workstation board	quality AM5 workstation board	ASUS/Gigabyte/ASRock sponsorship, open-box
Memory	256 GB ECC RDIMM	128 GB RAM	Kingston/Micron/enterprise surplus
System drive	2 TB high-endurance NVMe	1–2 TB NVMe	POSM purchase
Work drive	4 TB NVMe	2 TB NVMe	storage sponsor
Artifact/cache	4 TB NVMe	secondary SSD	storage sponsor or later addition
Backup	8–16 TB separate target	external HDD	Seagate/WD or POSM purchase
GPU	16–24 GB VRAM	no GPU initially	AMD/NVIDIA/surplus
Network	10 GbE	2.5 GbE	sponsor or campus surplus
PSU	1000–1300 W high-quality	platform-appropriate	POSM purchase
UPS	1500 VA or greater	reused chassis with new batteries	APC/Eaton/CyberPower/surplus
Cooling	full-surface TR5 cooler	AM5 cooler for fallback	Noctua/be quiet!/POSM

Case	secure high-airflow workstation case	existing rack/tower	Fractal/be quiet!/POSM
Watchdog	Raspberry Pi plus sensors	small x86/SBC	DigiKey or POSM

G.2 POSM cash plan

With a maximum contribution near $1000, prioritize PSU, boot NVMe, cooler, cabling, and verified backup/UPS components. Delay cosmetic upgrades and optional GPU. If no anchor CPU/platform donation appears, a 9950X-class build may require reallocation or additional fundraising.

G.3 Secondary node strategy

A donated retired workstation with at least 64–128 GB RAM and modern x86 virtualization support can become a useful worker node. Standardize a secure wipe, hardware test, BIOS update, disk health check, memory test, and qFoundry runner enrollment procedure.

G.4 Sponsor deliverables

Each sponsor receives the agreed factual acknowledgment, component photos, project description, use statistics, student impact, and final report. Sponsorship language shall not imply technical endorsement or control unless explicitly agreed.

Chapter H

Sentinel Research Models and Evaluation

H.1 Model registry fields

Each statistical model is registered with identifier, semantic version, owner, training query, features, priors, likelihood, inference method, validation split, calibration metrics, deployment authority, and rollback version.

H.2 Hierarchical cost model

A candidate log-cost model is:

$$\log C_i \sim \mathcal{N}(\mu_i, \sigma^2_{g(i)}), \tag{H.1}$$

$$\mu_i = \alpha + u_{\text{repo}[i]} + v_{\text{task}[i]} + w_{\text{model}[i]} + \mathbf{x}_i^\top \boldsymbol{\beta}, \tag{H.2}$$

with hierarchical priors on repository, task, and model effects. Heavy-tailed alternatives should be evaluated because occasional agent failures create extreme costs.

H.3 Completion survival model

Time-to-milestone may use an accelerated failure-time or proportional-hazards formulation with right-censoring for paused projects. Human waiting time is modeled separately from active compute time.

H.4 Latent progress model

A hidden semi-Markov model uses emissions such as passing-test count, requirement evidence, diff novelty, command repetition, reversion, and time since checkpoint. States and duration distributions are validated against human-labeled project traces.

H.5 Changepoint model

Bayesian online changepoint detection maintains a posterior over the current run length [1]. Candidate streams include cost per minute, failures per worker turn, tests passed per dollar, and cache-hit rate.

H.6 Routing bandit

The context includes task type, repository language, test health, current failure count, budget, deadline, and previous model outcomes. Actions are routing policies rather than only model names. Rewards combine verified success, cost, elapsed time, human burden, security penalties, and rework. Knapsack constraints represent finite budget and compute [2].

H.7 Safe optimization

Parameters that affect safety or cost begin in a known safe region. Safe Bayesian optimization proposes bounded candidates while respecting constraint confidence. No learned optimizer controls red permissions or hard budgets.

H.8 Calibration

For probability forecasts, report reliability diagrams, Brier score, log score, calibration intercept and slope, and subgroup sample size. For intervals, report empirical coverage and width. Conformal methods may wrap predictive models to improve coverage under limited assumptions [4, 16].

H.9 Sequential experiments

Improvement experiments define null and alternative hypotheses, primary metric, guardrail metrics, budget, maximum sample, stopping rule, and multiple-testing policy. Sequential probability tests or Bayesian decision thresholds may stop early when evidence is decisive.

H.10 No-data regime

At launch, models use conservative priors and broad uncertainty. Rules retain authority. Sentinel must display “insufficient comparable data” rather than inventing a narrow interval. Adaptive policies remain shadow-only until validation thresholds are met.

Chapter I

Glossary

Term	Definition
Cell	One project supervisor, Claude worker, contract, budget, sandbox, and event stream.
Contract	Versioned human-approved specification governing one project.
Contract compiler	System that transforms the human contract into structured requirements, acceptance plans, and Claude execution packs.
Credential broker	Service that performs authenticated actions without exposing raw credentials to models.
Digital twin	Monte Carlo or probabilistic simulation of portfolio schedules, costs, failures, and human response.
Event	Immutable structured record of an action, decision, observation, or state change.
Green permission	Deterministically approved low-risk operation.
Hard ceiling	Absolute monetary amount beyond which a request may not be initiated.
Meaningful progress	New verified evidence that reduces remaining contract obligations or uncertainty.
Model gateway	Local service controlling provider authentication, budgets, routing, redaction, and usage.
Ordinary user	Approved POSM user with project-scoped access and one active project.
Preflight quote	Time-, cost-, risk-, and confidence-aware execution plan created before meaningful work.
Red permission	Action requiring master or secure human confirmation.
Salvage mode	Graceful low-budget shutdown and resumable handoff procedure.
Sentinel	Portfolio auditing, statistical intelligence, security, scheduling, and improvement subsystem.
Supervisor	OpenAI model responsible for discovery, task direction, verification review, and escalation.

Technically ready	Supervisor and verification system conclude that the approved contract is satisfied.
User accepted	Project owner accepts the technically ready deliverable.
Worker	Claude Sonnet or Opus coding agent operating inside the project sandbox.
Yellow permission	Operation that may be approved by a delegated supervisor or project owner.

Chapter J

Initial Decision Log

ID	Decision	Rationale
ADR-001	Product name is POSM qFoundry	communicates an engineering foundry while preserving POSM's q naming language
ADR-002	OpenAI supervises; Claude codes	aligns trusted judgment with preferred coding model
ADR-003	contract-first execution	prevents drift and makes completion testable
ADR-004	event-driven supervisors	prevents idle token burn
ADR-005	fifth Cell is Sentinel	separates building work from audit, security, and improvement
ADR-006	raw telemetry immutable	Sentinel cannot alter its own evidence
ADR-007	initial POSM project cap is $30	limits financial exposure while testing
ADR-008	one active project per ordinary user	fair shared-resource governance
ADR-009	master may run multiple projects	supports POSM portfolio administration
ADR-010	no automatic merge to main	preserves human ownership of protected branches
ADR-011	two-step merge confirmation	prevents ambiguous natural-language approval
ADR-012	approved registries only	reduces supply-chain and arbitrary network risk
ADR-013	Windows desktop, Linux workers	polished local client with reproducible execution
ADR-014	eventual Linux lab backend	improves reliability and multi-user isolation
ADR-015	public software uses operator keys	POSM does not subsidize public usage
ADR-016	private-first, then open source	allows validation before public distribution
ADR-017	dedicated mailbox with reply ingestion	user responds reliably to email
ADR-018	email 8 AM–11 PM, max ten/day	balances urgency and attention

ADR-019	POSM contributes at most about $1000 to first computer	drives sponsorship and mixed-node strategy
ADR-020	Raspberry Pi is watchdog, not main cluster	avoids ARM and low-memory constraints

J.1 Future decisions

New architecture decisions shall be recorded as ADR documents and summarized in this appendix at each specification release.

References

[1] Ryan Prescott Adams and David J. C. MacKay. "Bayesian Online Changepoint Detection". In: *arXiv preprint arXiv:0710.3742* (2007). URL: https://arxiv.org/abs/0710.3742.

[2] Shipra Agrawal, Nikhil R. Devanur, and Lihong Li. "An Efficient Algorithm for Contextual Bandits with Knapsacks, and an Extension to Concave Objectives". In: *Proceedings of the 29th Conference on Learning Theory*. Vol. 49. Proceedings of Machine Learning Research. 2016, pp. 4–18. URL: https://proceedings.mlr.press/v49/agrawal16.html.

[3] AMD. *AMD University Program Donation Program*. 2026. URL: https://www.amd.com/en/corporate/university-program/donation-program.html (visited on 06/14/2026).

[4] Anastasios N. Angelopoulos, Stephen Bates, Michael I. Jordan, and Jitendra Malik. "Conformal Risk Control". In: *International Conference on Learning Representations* (2024). URL: https://arxiv.org/abs/2208.02814.

[5] Anthropic. *Claude Agent SDK Overview*. 2026. URL: https://docs.anthropic.com/en/docs/claude-code/sdk (visited on 06/14/2026).

[6] Anthropic. *Claude Code LLM Gateway Configuration*. 2026. URL: https://docs.anthropic.com/en/docs/claude-code/llm-gateway (visited on 06/14/2026).

[7] DigiKey. *DigiKey Academic Program*. 2026. URL: https://www.digikey.com/en/edu (visited on 06/14/2026).

[8] Docker. *Resource Constraints*. 2026. URL: https://docs.docker.com/engine/containers/resource_constraints/ (visited on 06/14/2026).

[9] Docker. *Rootless Mode*. 2026. URL: https://docs.docker.com/engine/security/rootless/ (visited on 06/14/2026).

[10] Christian Fiedler et al. "On Safety in Safe Bayesian Optimization". In: *arXiv preprint arXiv:2403.12948* (2024). URL: https://arxiv.org/abs/2403.12948.

[11] GitHub. *Generating an Installation Access Token for a GitHub App*. 2026. URL: https://docs.github.com/en/apps/creating-github-apps/authenticating-with-a-github-app/generating-an-installation-access-token-for-a-github-app (visited on 06/14/2026).

[12] GitHub. *Permissions Required for GitHub Apps*. 2026. URL: https://docs.github.com/en/rest/authentication/permissions-required-for-github-apps (visited on 06/14/2026).

[13] Google. *Configure Push Notifications in the Gmail API*. 2026. URL: https://developers.google.com/workspace/gmail/api/guides/push (visited on 06/14/2026).

[14] Google. *Implement Server-Side Authorization for Gmail API*. 2026. URL: https://developers.google.com/workspace/gmail/api/auth/web-server (visited on 06/14/2026).

[15] Google. *Manage Threads in the Gmail API*. 2026. URL: https://developers.google.com/workspace/gmail/api/guides/threads (visited on 06/14/2026).

[16] Bror Hultberg, Dave Zachariah, and Antônio H. Ribeiro. "Anytime-Valid Conformal Risk Control". In: *arXiv preprint arXiv:2602.04364* (2026). URL: https://arxiv.org/abs/2602.04364.

[17] Microsoft. *Get Started with Docker Remote Containers on WSL 2*. 2024. URL: https://learn.microsoft.com/en-us/windows/wsl/tutorials/wsl-containers (visited on 06/14/2026).

[18] OpenAI. *Batch API*. 2026. URL: https://developers.openai.com/api/docs/guides/batch (visited on 06/14/2026).

[19] OpenAI. *Prompt Caching*. 2026. URL: https://developers.openai.com/api/docs/guides/prompt-caching (visited on 06/14/2026).

[20] OpenAI. *Tracing — OpenAI Agents SDK*. 2026. URL: https://openai.github.io/openai-agents-python/tracing/ (visited on 06/14/2026).

[21] OpenAI. *Usage — OpenAI Agents SDK*. 2026. URL: https://openai.github.io/openai-agents-python/usage/ (visited on 06/14/2026).

[22] Yanan Sui, Alkis Gotovos, Joel Burdick, and Andreas Krause. "Safe Exploration for Optimization with Gaussian Processes". In: *Proceedings of the 32nd International Conference on Machine Learning*. Vol. 37. Proceedings of Machine Learning Research. 2015, pp. 997–1005. URL: https://proceedings.mlr.press/v37/sui15.html.

[23] Tauri Programme. *Tauri 2.0*. 2026. URL: https://v2.tauri.app/ (visited on 06/14/2026).

[24] Tauri Programme. *Tauri Capabilities*. 2026. URL: https://v2.tauri.app/security/capabilities/ (visited on 06/14/2026).

[25] Temporal Technologies. *Human-in-the-Loop AI Agent*. 2026. URL: https://docs.temporal.io/ai-cookbook/human-in-the-loop-python (visited on 06/14/2026).

[26] Temporal Technologies. *Temporal Workflows*. 2026. URL: https://docs.temporal.io/workflows (visited on 06/14/2026).

[27] University of Minnesota Facilities Management. *About the ReUse Program*. 2026. URL: https://facilities.umn.edu/our-services/waste-recycling-reuse/reuse/about-reuse-program (visited on 06/14/2026).